Data pre-processing

Assoc. Prof. Kitsana Waiyamai, Ph. D.

Dept. Of Computer Engineering
Faculty of Engineering, Kasetsart University
Bangkok, Thailand



Reality of Real-World Data

- Raw data is rarely perfect. It is often:
  - **Incomplete:** Lacking attribute values or certain interests.
  - **Noisy:** Containing **noises** or **outliers**.
  - **Inconsistent:** Diff in data formats or naming conventions.
    - **Example**:
      - **Address**: "BKK", "Bangkok", "Krung Thep".
      - **Date Formats**: DD/MM/YYYY vs. MM-DD-YY.
      - **Logic Errors**: Age is 20, but the birth year is 1980.
    - **Reasons** or **Causes**: Merging data from multiple sources without standardization => **Impact**: Makes grouping and aggregation impossible.
  - There are also
    - Fake data
    - Duplicate data
    - Etc.
- Preprocessing
  - is often the most time-consuming yet critical step in the analytical process.

Data Pre-processing

- **After this session, you will be able to:**
- **Understand Data Quality:**
  - Grasp the "Garbage In, Garbage Out" (GIGO) principle and identify real-world data issues.
- **Handle Noisy Data:**
  - Explain and apply techniques for Data Cleaning.
- **Apply Data Transformation:**
  - Understand how to use Normalization to prepare data for modeling.
- **Master Data Reduction:**
  - Differentiate between Dimensionality and Numerosity reduction to improve efficiency.

"Garbage In, Garbage Out" (GIGO)

- **Why Preprocessing Matters:**
- The quality of the **input** determines the quality of the **output**.
- No matter how sophisticated your algorithm is, poor data quality leads to:
  - Inaccurate predictions.
  - Unreliable insights.
- Flawed business decisions.

Poor data quality negatively affects model performance

- **Example**: a classification model for detecting people who are loan risks is built using **poor data**
  - Some credible applicants are denied loans.
  - Provide loans to people who default on debt repayment
- **Pre-processing is needed** to
  - Enhance the **quality of the data**.
  - Reduce the impact of incomplete and inconsistent data and

Some Pre-processing Techniques

- Selection and Integration of Relevant Data
- Sampling Data (Over-Sampling VS Under-Sampling)
- Aggregation & Generalization
- Data Cleansing
  - Handling missing values
  - Removing noises and outliers
  - De-duplicating
  - Etc.
- Data Transformation
  - Categorical => Numerical
    - Encoding
  - Numerical => Categorical
    - Discretization
- Feature Reduction
  - Feature subset selection
  - Feature creation & extraction
- Feature Scaling
  - Normalization and Standardization
- Etc.

Data pre-processing often consumes 80% or more of the effort in a Big Data project

Incomplete Data

- **Definition: Data without variable values (missing value)**
- **Reasons** or **Causes**:
  - Equipment malfunction,
  - human error during entry,
  - users refusing to provide sensitive info,
  - Information is not collected
    (e.g., people decline to provide sensitive info such as their age and weight)
  - Attributes may not be applicable to all cases
    (e.g., annual income is not applicable to children)
- **Impact**:
  - Creates statistical Bias;
  - Most algorithms cannot process null values

Why missing values treatment is required?

- Reduce the power / fit of a model
- Lead to a biased model
  - Wrong prediction / classification
- **Handling Missing Values is requited !**
  - Delete rows? Delete columns?
    - Not that easy to delete because we lost data !
  - Imputation => Replace values

- **Bad treatment of missing values**: column for 'Gender' if highly correlated with Play Cricket

Name	Weight	Gender	Play Cricket/ Not
Mr. Amit	58	M	Y
Mr. Anil	61	M	Y
Miss Swati	58	F	N
Miss Richa	55	?	Y
Mr. Steve	55	M	N
Miss Reena	64	F	Y
Miss Rashmi	57	?	Y
Mr. Kunal	57	M	N

Handling Missing Values

Data with no missing value treatment

Name	Weight	Gender	Play Cricket/ Not
Mr. Amit	58	M	Y
Mr. Anil	61	M	Y
Miss Swati	58	F	N
Miss Richa	55		Y
Mr. Steve	55	M	N
Miss Reena	64	F	Y
Miss Rashmi	57		Y
Mr. Kunal	57	M	N

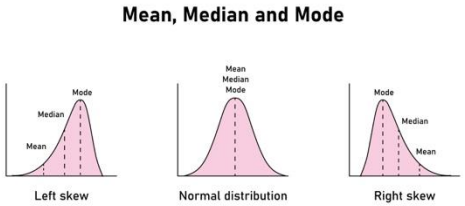
Gender	#Students	#Play Cricket	%Play Cricket
F	2	1	50%
M	4	2	50%
Missing	2	2	100%

Data after missing value treatment

Name	Weight	Gender	Play Cricket/ Not
Mr. Amit	58	M	Y
Mr. Anil	61	M	Y
Miss Swati	58	F	N
Miss Richa	55	F	Y
Mr. Steve	55	M	N
Miss Reena	64	F	Y
Miss Rashmi	57	F	Y
Mr. Kunal	57	M	N

Gender	#Students	#Play Cricket	%Play Cricket
F	4	3	75%
M	4	2	50%

Statistical Imputation



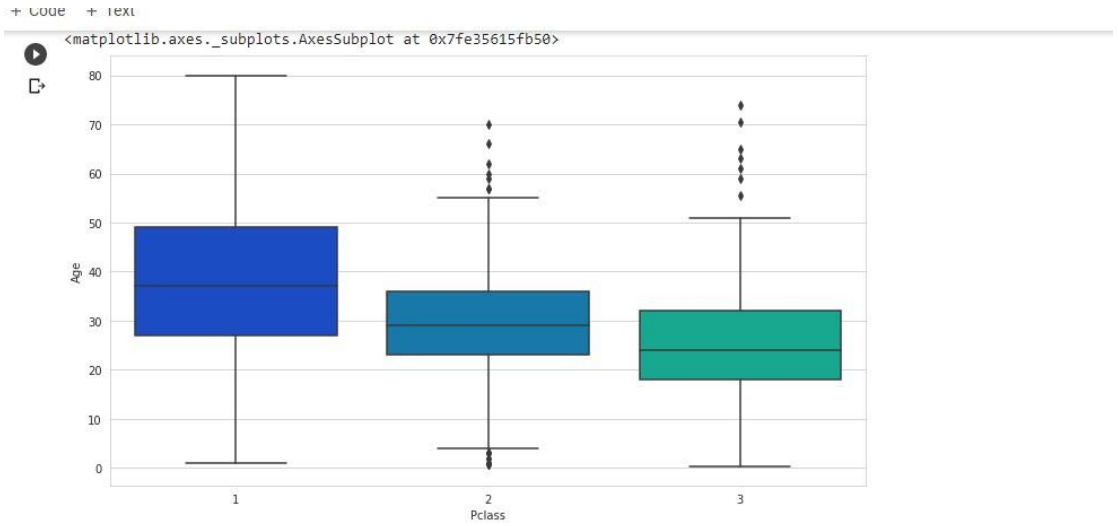
- **Manually fill in the missing value**: time consuming
- **Filling missing fields with certain values**
  - Use a global constant to fill in the missing value:
  - For example, “MISSING”, simple, but can mislead the mining techniques.
  - 0, or the empty string, etc.

- **Use a measure of central tendency** for the attribute to fill in the missing value.
- **Statistical measures** – mean, median, mode
  - **Mean** (Average): Best for Numerical data =>Highly sensitive to Outliers.
  - **Median**: Best for Numerical data with a skewed distribution => More Robust than the mean.
  - **Mode**: Best for Categorical (Nominal) data.
- **Use the attribute mean or median for all samples belonging to the same class** as the given object.

Example, if a day in July is missing its temperature value, fill it with the median temperature in July

Missing values treatment: Example

Row No.	PassengerId	Survived	Pclass	Name	Sex	Age ↓
6	6	0	3	Moran, Mr. J...	male	?
18	18	1	2	Williams, Mr...	male	?
20	20	1	3	Masselmani,...	female	?
27	27	0	3	Emir, Mr. Fa...	male	?
29	29	1	3	O'Dwyer, Mi...	female	?
30	30	0	3	Todoroff, M...	male	?
32	32	1	1	Spencer, Mr...	female	?
33	33	1	3	Glynn, Miss....	female	?
37	37	1	3	Mamee, Mr....	male	?
43	43	0	3	Kraeff, Mr. ...	male	?
46	46	0	3	Rogers, Mr. ...	male	?



Missing values treatment

- **Use frequent values**
  - For categorical attributes => lead to imbalanced problem !
- **Derive values from other attributes**
  - For example, use domain knowledge
- **Estimate missing values**
  - Use the most probable value to fill in the missing value
  - Apply other learning techniques to predict missing
- …

Missing value treatment with Python

isnull.any()	Check if there are any NaN values in each attribute
isnull.sum()	Count how many records have NaN values for each attribute
dropna()	Drop records with NaN values
fillna()	Replace NaN values with a specified value (many opts)

```
import pandas as pd
import numpy as np

df = pd.read_csv(r'data\test-missing-values.csv')
#display(df)

print(df.isnull().any())
print()
print(df.isnull().sum())

#กรณีที่ต้องการตรวจสอบเพียงบางคอลัมน์ เช่น
#print(df['volume'].isnull().any())
#print(df[['volume', 'weight']].isnull().sum())

df.dropna(inplace=True)  #inplace=True คือให้แทนที่ผลลัพธ์ในออบเจ็กต์เดิม
display(df)
```

Missing value treatment with Python

isnull.any()	Check if there are any NaN values in each attribute
isnull.sum()	Count how many records have NaN values for each attribute
dropna()	Drop records with NaN values
fillna()	Replace NaN values with a specified value (many opts)

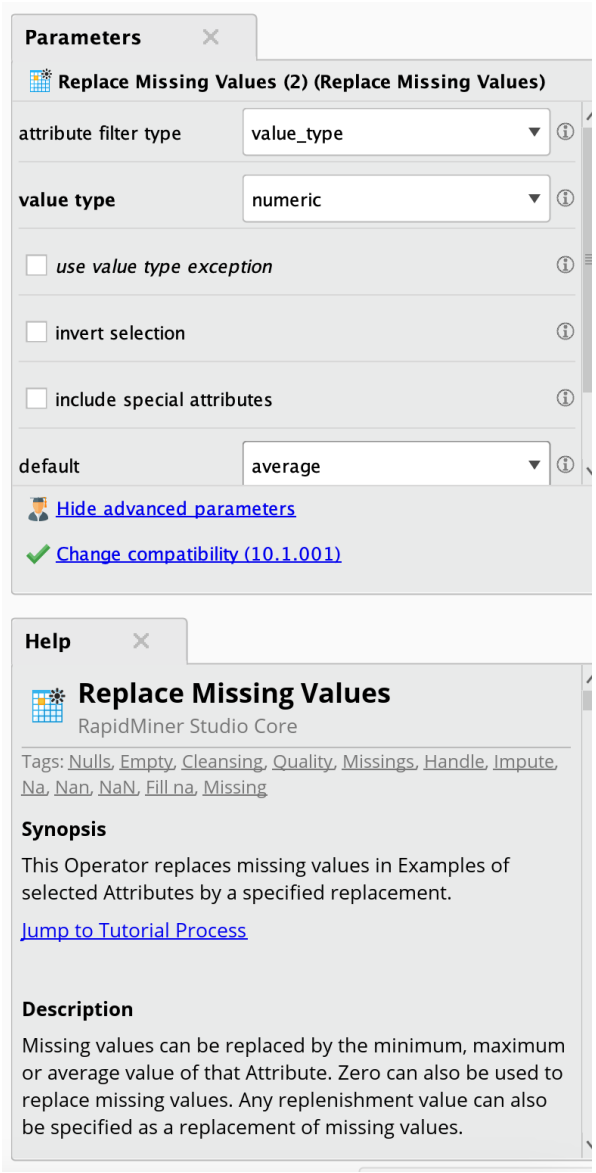
```
[ ]  #missing value (fillna-1)

     v_mean = df['volume'].mean()
     df['volume'].fillna(v_mean, inplace=True)
     df['weight'].fillna(df['weight'].mean(), inplace=True)
```

```
[ ]  #missing value (fillna-2)

     df['volume'].fillna(method='ffill', inplace=True) # Takes the value from the previous non-null cell and fills it with the next cell that contains the null
     df['weight'].fillna(method='bfill', inplace=True)  # Takes the value from the nerxt non-null cell and fills it with the previous cell that contains the null
```

Missing value treatment with RapidMiner



- **none**: Missing values are not replaced by default.
- **minimum**: Missing values are replaced by the minimum value of that Attribute.
- **maximum**: Missing values are replaced by the maximum value of that Attribute.
- **average**: Missing values are replaced by the average value of that Attribute.
- **zero**: Missing values are replaced by zero.
- **value**: Missing values are replaced by the value specified in the replenishment value parameter.

Model-Based Imputation

Using **Intelligence to Fill Gaps**:

- **Concept**: Treat the missing attribute as a Target Variable.
- **Method**: Use Regression or Machine Learning to predict the missing value based on other features.
- **Advantage**: Preserves relationships between variables much better than simple mean/median imputation.
- **Note**: Filling with a mean can lead to Underestimated Variance because it ignores the uncertainty of the missing data.

Noisy Data: Noise and Outliers

Noise

- Noise refers to **small**, **random deviations** from the true underlying signal.
- Noise **hides patterns** in data but is not the true signal.
- Noise may be **valid** or **invalid**, depending on the data context.

Reasons / Causes

- Faulty or imprecise sensors
- Measurement limitations
- Human data entry inaccuracies
- Transmission or environmental interference

Impact

- Increases data variability
- Distorts statistical metrics such as **Mean** and **Standard Deviation**
- Causes models to learn from **irrelevant fluctuations** rather than true patterns

Real-world Examples

- Slight fluctuations in temperature sensor readings
- Minor errors in recorded exam scores
- Distortion of a person's voice on a poor phone connection
- "Snow" on a television screen

Noise Interpretation

- **For attributes**: Noise refers to small modifications of original values
- **For objects**: Noise appears as **extraneous (irrelevant)** objects

Handling Noisy Data

- The Danger: Noise causes Overfitting, where the model learns the "error" rather than the "pattern."
- Techniques for Smoothing:
  - Binning: Local smoothing by grouping.
  - Regression: Fitting data to a function.
  - Clustering: Detecting structural outliers.

Smoothing by Binning

- **Concept**: Sort data and partition it into "bins." Replace values in the bin with a summary statistic.
- **Objective**: Reduces the impact of minor fluctuations and highlights general trends.
- **Methods**:
  - Smooth by Mean: Replace all values in a bin with the bin's mean.
  - Smooth by Boundaries: Replace values with the closest "edge" (min or max) of the bin.

Smoothing data for reduce noise

- Binning methods smooth a sorted data value by consulting its “neighbor-hood,” that is, the values around it.
- For example, smoothing by **bin means** => **each value in a bin is replaced by the mean value of the bin**.

Sorted data for *price* (in dollars): 4, 8, 15, 21, 21, 24, 25, 28, 34

Partition into (equal-frequency) bins:
Bin 1: 4, 8, 15
Bin 2: 21, 21, 24
Bin 3: 25, 28, 34

Smoothing by bin means:
Bin 1: 9, 9, 9
Bin 2: 22, 22, 22
Bin 3: 29, 29, 29

Smoothing by bin boundaries:
Bin 1: 4, 4, 15
Bin 2: 21, 21, 24
Bin 3: 25, 25, 34

- Other methods
  - **Smoothing by bin medians** can be employed, in which each bin value is replaced by the bin median.
  - **Smoothing by bin boundaries**, the minimum and maximum values in a given bin are identified as the bin boundaries. Each bin value is then replaced by the closest boundary value. In general, the larger the width, the greater the effect of the smoothing

Noisy Data: Noise and Outliers

Outiliers

- Outliers are data points that deviate significantly from the majority of the data.
- They are clearly distant from the general trend.

Reasons / Causes

- Data entry mistakes
- Measurement or recording errors
- Rare but real events
- System malfunctions

Impact

- Strongly distort statistical metrics such as Mean and Standard Deviation
- Can dominate model training and bias predictions
- May mislead analysis if not properly handled

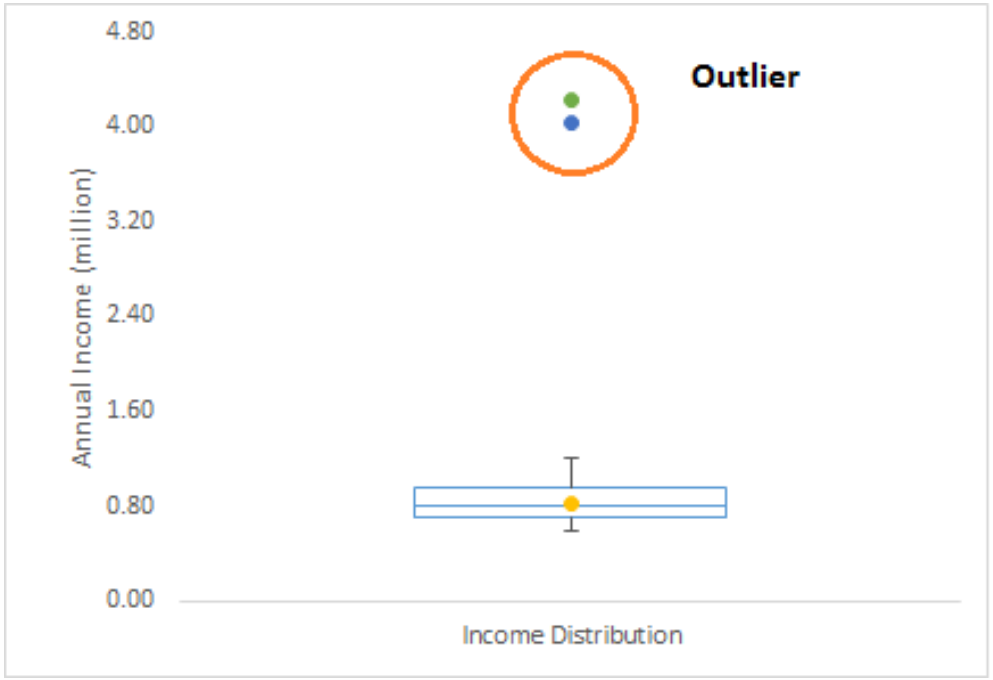
Examples

- Daily sales usually around 20,000 THB, but one record shows 5,000,000 THB
- Customer income values mostly below 100,000 THB, with one record at 10,000,000 THB

Important Note

- Outliers should not be removed automatically
- They must be analyzed using domain knowledge to determine whether they are meaningful

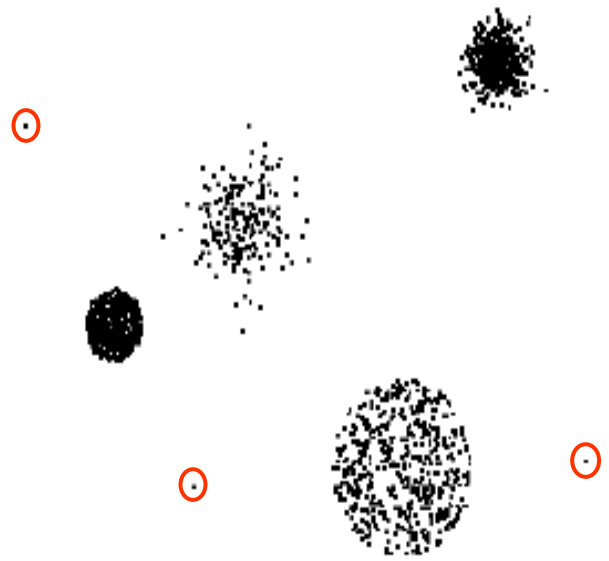
What is an Outlier?



Outlier is an observation that **appears far away** and **diverges** from an overall pattern in a sample.

Outliers

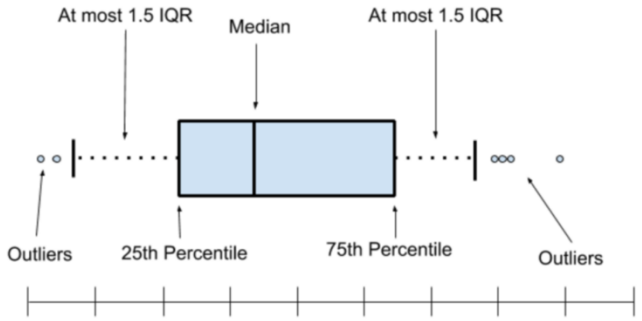
- Outliers (or Anomalies) are data that is unusual or different from most data
- Objects with characteristics that are considerably different than most of the other data objects in the data set
  - **Case 1:** Outliers are noise that interferes with data analysis
  - **Case 2:** Outliers are the goal of our analysis
    - Credit card fraud
    - Intrusion detection



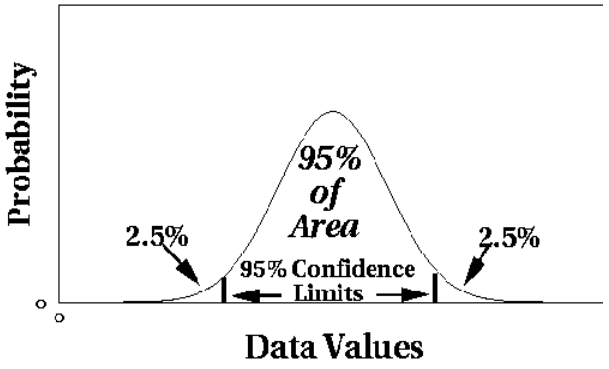
Example of Outliers

Without Outlier	With Outlier
4, 4, 5, 5, 5, 5, 6, 6, 6, 7, 7	4, 4, 5, 5, 5, 5, 6, 6, 6, 7, 7,300
Mean = 5.45	Mean = 30.00
Median = 5.00	Median = 5.50
Mode = 5.00	Mode = 5.00
Standard Deviation = 1.04	Standard Deviation = 85.03

Outliers Detection Techniques



- Graphical Approaches
- Statistical Approaches
- Proximity-based
  - Anomalies are points far away from other points
- Density-based Approaches
- Clustering-based
  - Points far away from cluster centers are outliers
  - Small clusters are outliers
- Other Approaches

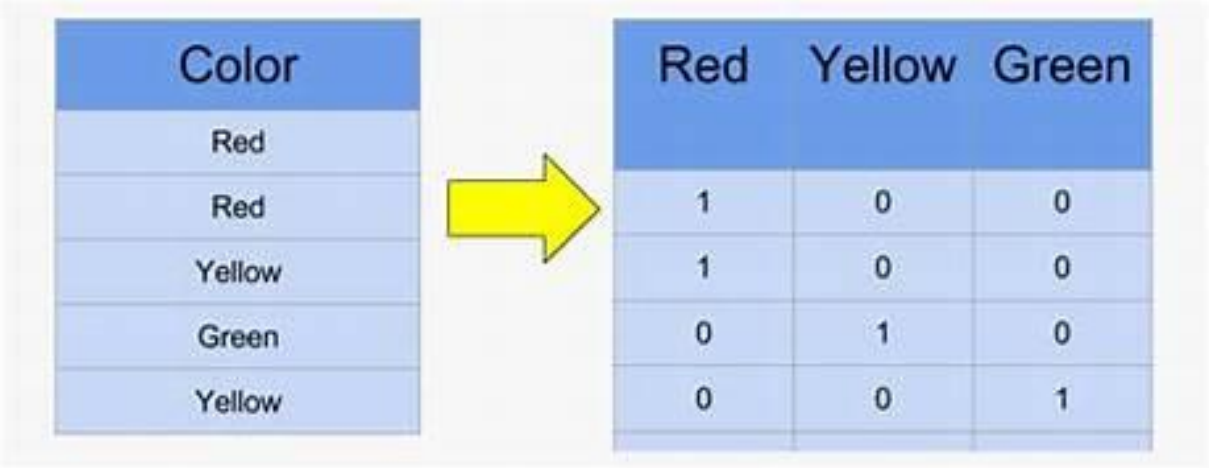


Data encoding: categorical =>numerical

- Data encoding, refers to the process of transforming categorical data into a numerical representation.
- Categorical Attributes => Numerical Attributes
- Various Types of Encoding
  - **One-hot-encoding**
    - Nominal => Numerical Attributes
  - **Label encoding**
    - Ordinal => Numerical Attributes
- One-hot-encoding with many categories
- Etc.

One hot encoding (dummy encoding)

- One-Hot- Encoding:
  - convert categorical variables with multiple categories into a binary vector representation.
  - Each category is transformed into a binary feature, where a value of 1 indicates the presence of that category and 0 indicates the absence.



Color
Red
Red
Yellow
Green
Yellow

Red	Yellow	Green
1	0	0
1	0	0
0	1	0
0	0	1

Color	Red	Yellow
Red	1	0
Red	1	0
Yellow	0	1
Green	0	0
Yellow	0	1

One hot encoding (dummy encoding)

ID	Purchased	Age	Salary	Country
1	No	44	72000	France
2	Yes	27	48000	Spain
3	No	30	54000	Germany
4	No	38	61000	Spain
5	Yes	40	63778	Germany
6	Yes	35	58000	France
7	No	39	52000	Spain
8	Yes	48	79000	France
9	No	50	83000	Germany
10	Yes	37	67000	France

nominal => numerical

Problem:
Curse of dimensionality !

ID	Purchased	Age	Salary	Country = France	Country = Spain	Country = Germany
2	Yes	-1.627	-1.364	0	1	0
3	No	-1.213	-0.846	0	0	1
4	No	-0.110	-0.240	0	1	0
6	Yes	-0.524	-0.500	1	0	0
7	No	0.028	-1.018	0	1	0
8	Yes	1.268	1.316	1	0	0
9	No	1.544	1.662	0	0	1
10	Yes	-0.248	0.279	1	0	0

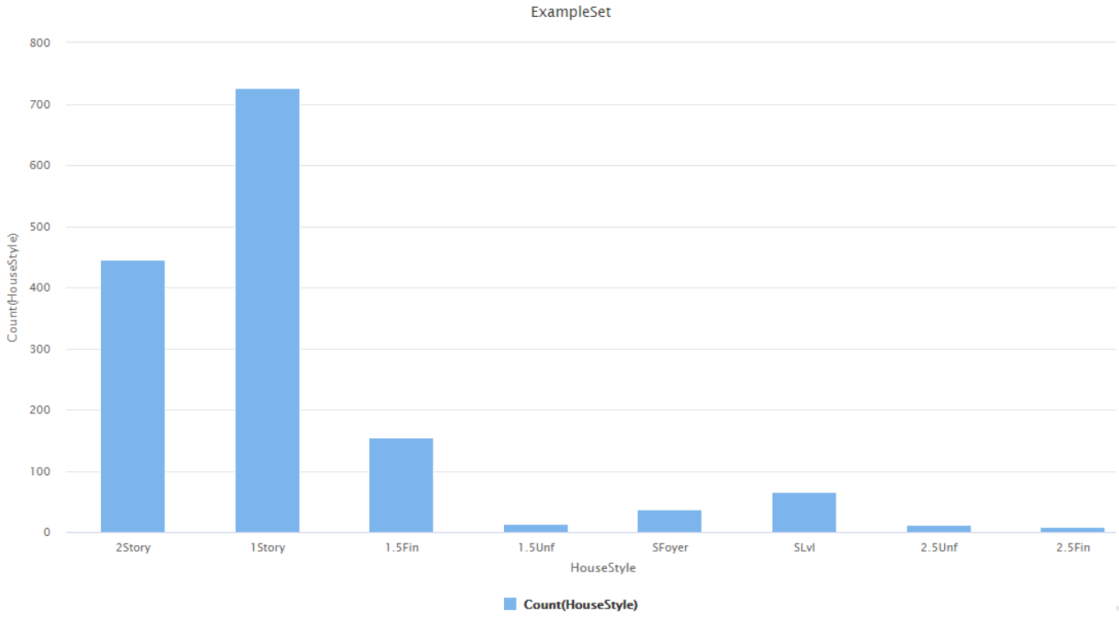
One hot encoding (dummy encoding)

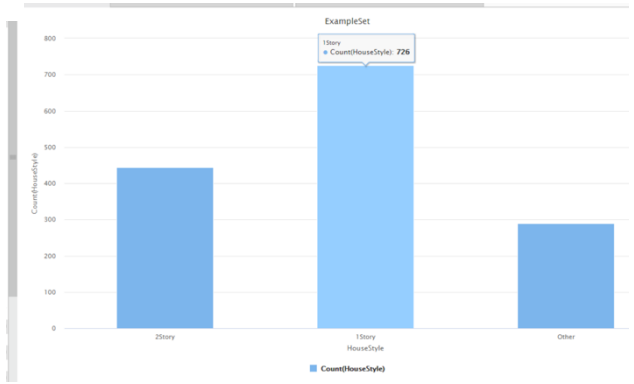
Conversion of a categorical attribute to three binary attributes.

Categorical Value	Integer Value	x_1	x_2	x_3
awful	0	0	0	0
poor	1	0	0	1
OK	2	0	1	0
good	3	0	1	1
great	4	1	0	0

Large number of categories => Combine categories

if a categorical attribute has a large number of categories, or some categories occur infrequently => reduce the number of categories by combining some of them.





Label-Encoding: Ordinal => Numerical

- Label- Encoding:
  - Label encoding assigns a unique numeric label to each category in a categorical variable.
  - Each category is mapped to a corresponding integer value.

Sex	Label
Male	1
Female	2
Other	3

Education	Label
Graduate	1
Masters	2
PHD	3

Label-Encoding with Python

```
from sklearn.preprocessing import LabelEncoder

data = ['Apple', 'Mango', 'Grape', 'Durian', 'Mango', 'Grape']
encoder = LabelEncoder()

encoder.fit(data)

enc_data1 = encoder.transform(data)
print(enc_data1)
#[0 3 2 1 3 2]

#ถ้าจะแปลงข้อมูลครั้งต่อไป (หากเป็นชุดเดียวกัน) ให้เรียกเมธอด transform() ได้เลย
enc_data2 = encoder.transform(['Mango', 'Durian', 'Durian'])
print(enc_data2)
#[3 1 1]

enc_data3 = encoder.transform(['Grape', 'Mango', 'Mango', 'Durian'])
print(enc_data3)
#[2 3 3 1]

enc_data4 = encoder.transform(['mango', 'Mangosteen', 'Melon'])  #Error
#เกิดข้อผิดพลาด เพราะ
#mango เขียนด้วยตัวพิมพ์ที่ไม่ตรงกับข้อมูลที่กำหนดให้แก่ fit()
#Mangosteen และ Melon ไม่มีอยู่ในข้อมูลที่กำหนดให้แก่ fit()
```

```
[0 3 2 1 3 2]
[3 1 1]
[2 3 3 1]
```

Encoding with RapidMiner

- **unique_integers**: If this option is selected, the values of nominal attributes can be seen as equally ranked, therefore the nominal attribute will simply be turned into a real valued attribute, the old values result in equidistant real values.
- **dummy_coding**: If this option is selected, for all values of the nominal attribute, excluding the comparison group, a new attribute is created. The comparison group can be defined using the comparison groups parameter. In every example, the new attribute which corresponds to the actual nominal value of that example gets value 1 and all other new attributes get value 0. If the value of the nominal attribute of this example corresponds to the comparison group, all new attributes are set to 0. Note that the comparison group is an optional parameter with 'dummy coding'. If no comparison group is defined, in every example the new attribute which corresponds to the actual nominal value of that example gets value 1 and all other new attributes get value 0. In this case, there will be no example where all new attributes get value 0.This can be easily understood by studying the attached example process.

Data Discretization: Numerical => Categorical

- **Discretization** transforms continuous numerical data into a categorical or discrete form.
- **Numerical Attributes => Categorical (Ordinal) Attributes**
- It involves dividing the continuous range of values into a set of intervals or bins and assigning each value to the corresponding bin.
- The goal of **discretization** is to reduce the number of values a continuous attribute assumes by grouping them into a number, n, of intervals (bins)

	name	age	binned
0	Alex	10	(0, 14]
1	Bob	16	(14, 24]
2	Clarke	26	(24, 64]
3	James	24	(14, 24]
4	John	69	(64, 100]
5	Michael	80	(64, 100]

Data Discretization

- Discretization is the process of converting a continuous attribute into an ordinal attribute
  - A potentially infinite number of values are mapped into a small number of categories
  - Discretization is often a required pre-processing step for many supervised learning methods.
  - Discretization is used in both unsupervised and supervised settings

Discretization Algorithms

Discretization algorithms can be divided into:

- **unsupervised vs. supervised**
- Unsupervised discretization algorithms do not take the class information into consideration
  - Faster and easier to implement
  - More prone to misclassification errors
- Equal Width
  - Divides data set into intervals of equal width
- Equal Frequency
  - Divides data set into intervals of equal frequency (number of data points in interval)

Supervised vs. Unsupervised discretization

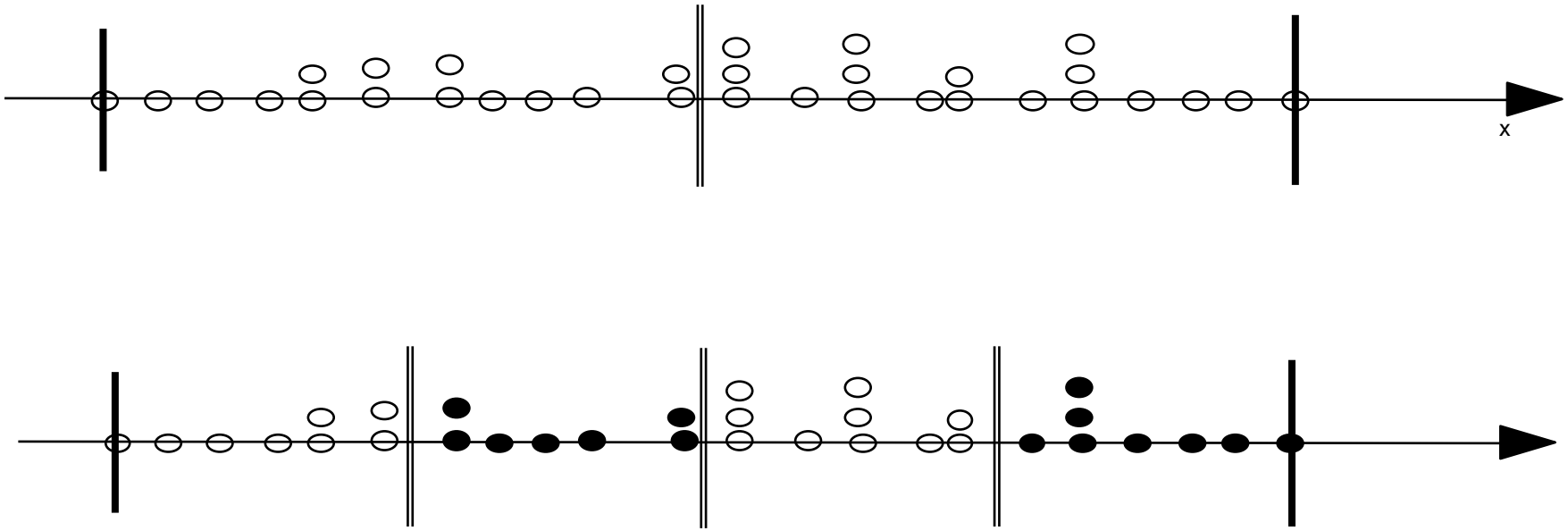
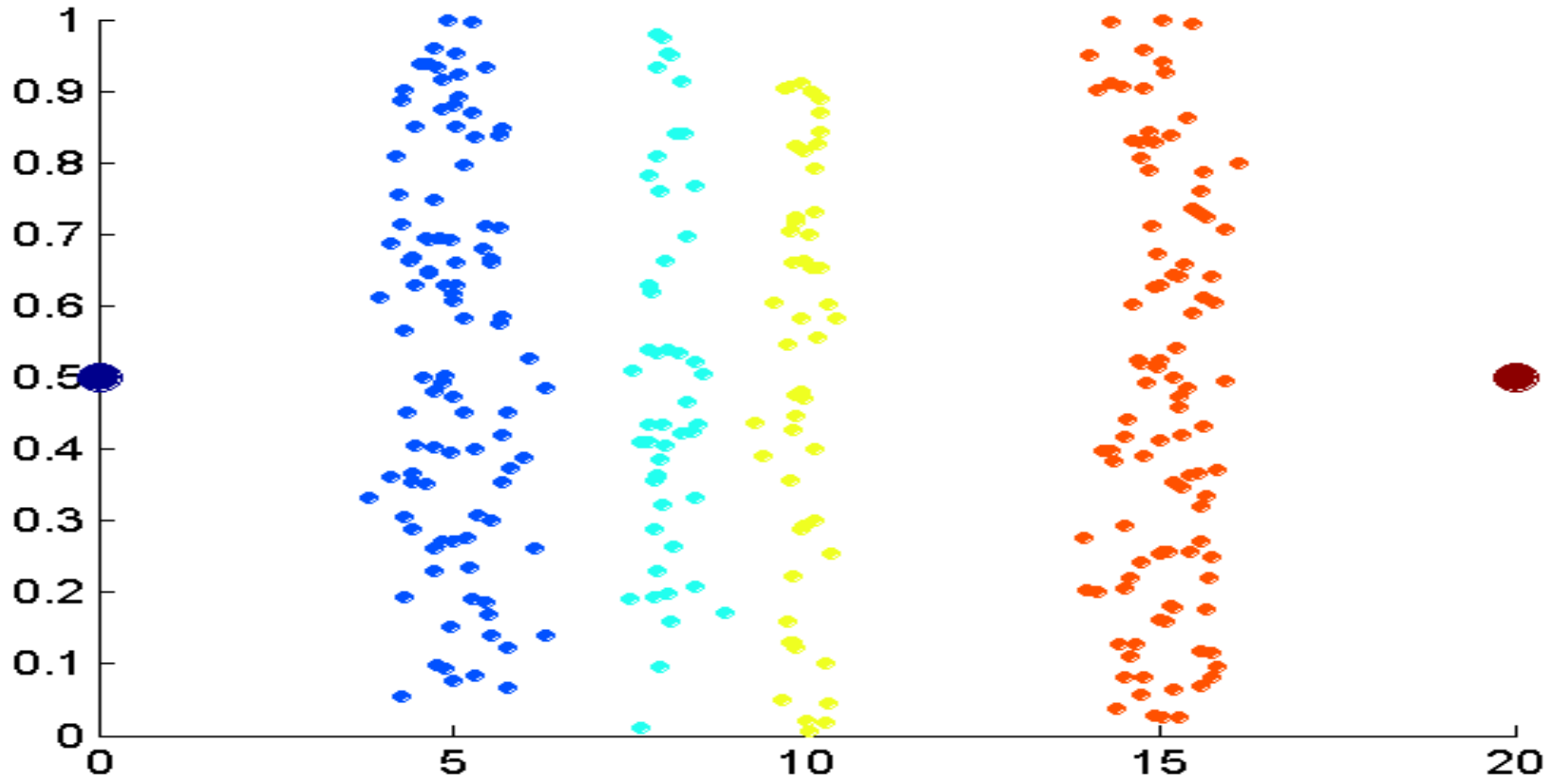


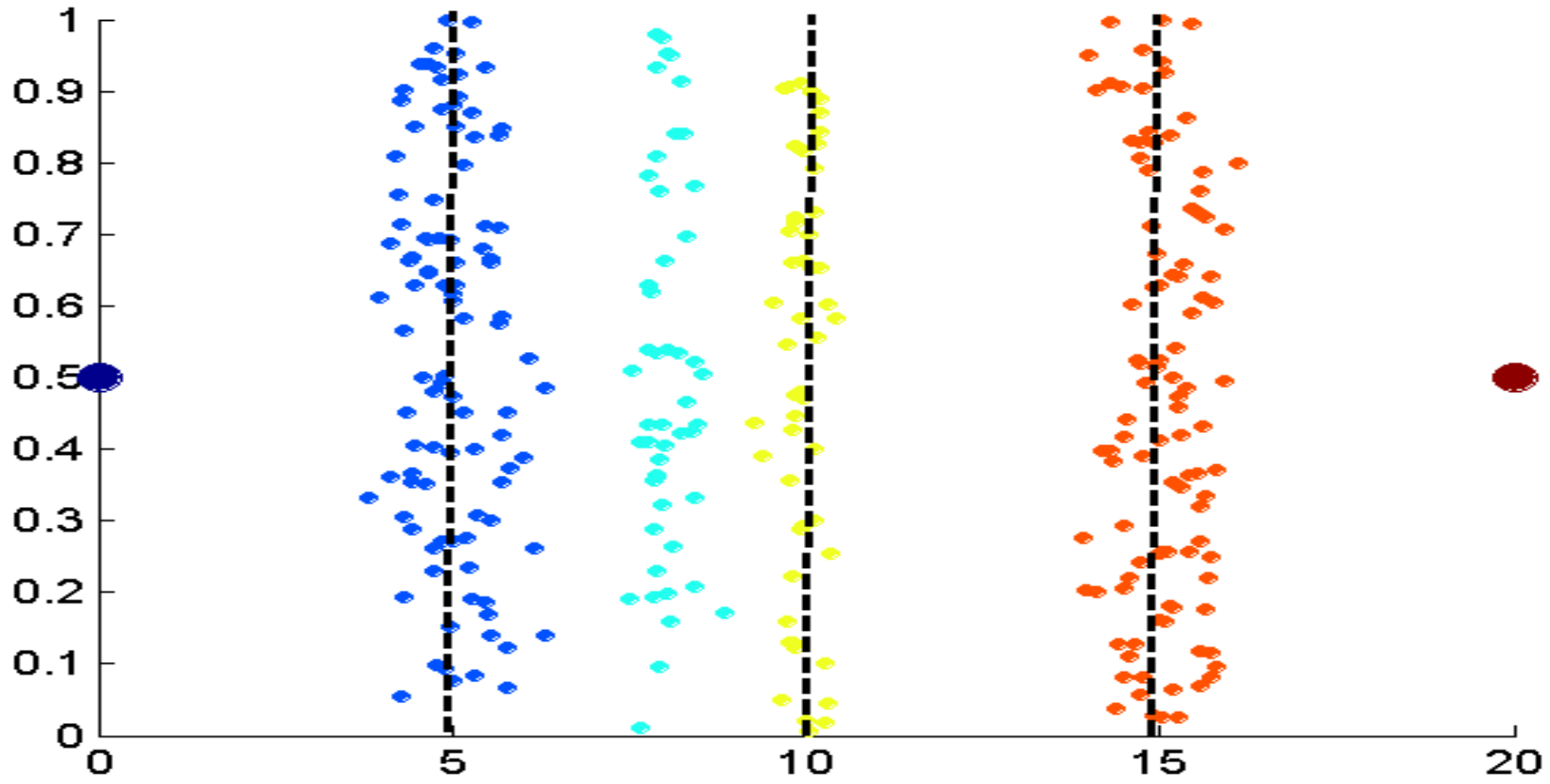
Illustration of the supervised vs. unsupervised discretization concept

Unsupervised Discretization



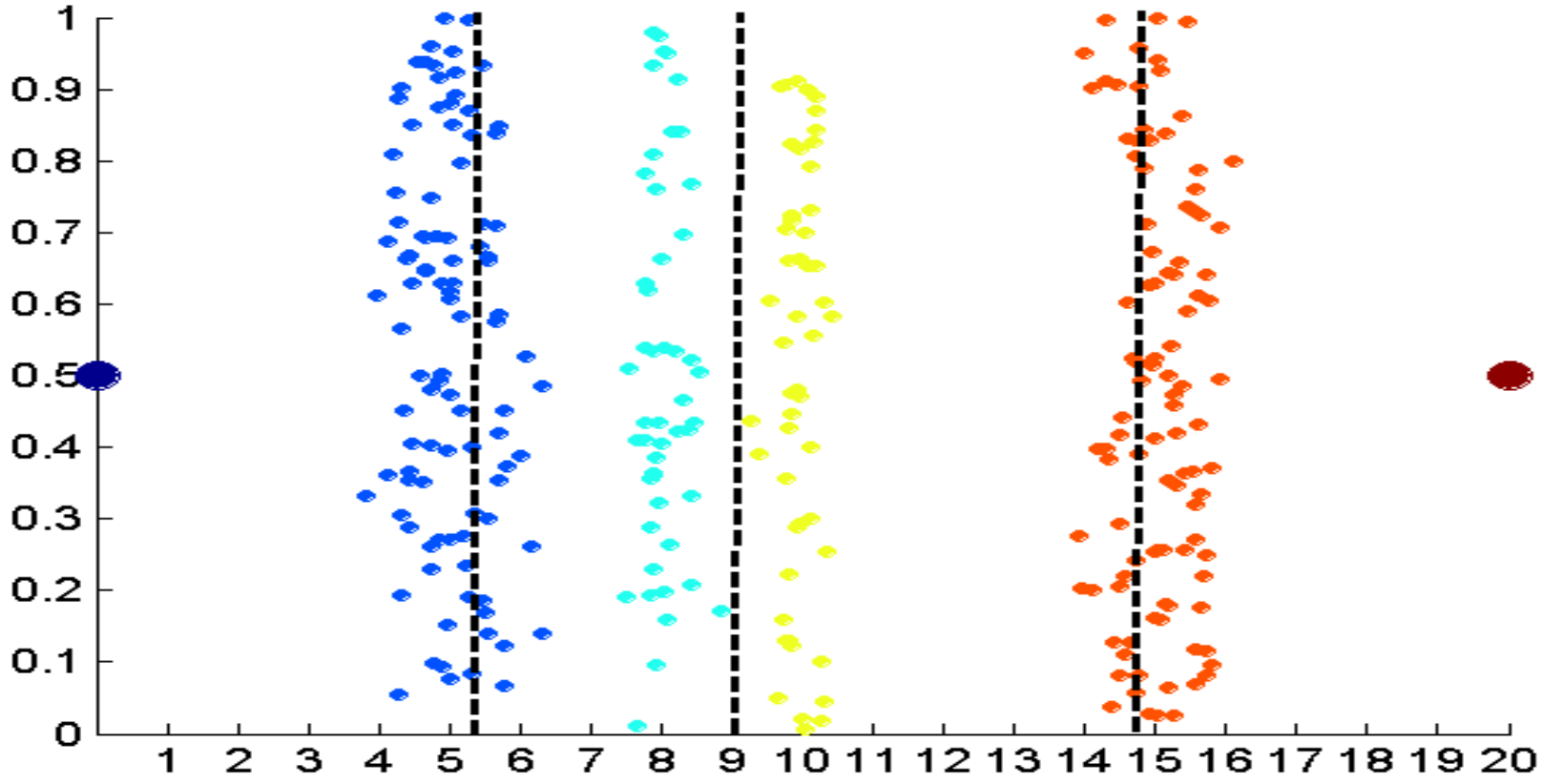
Data consists of four groups of points and two outliers.

Unsupervised Discretization



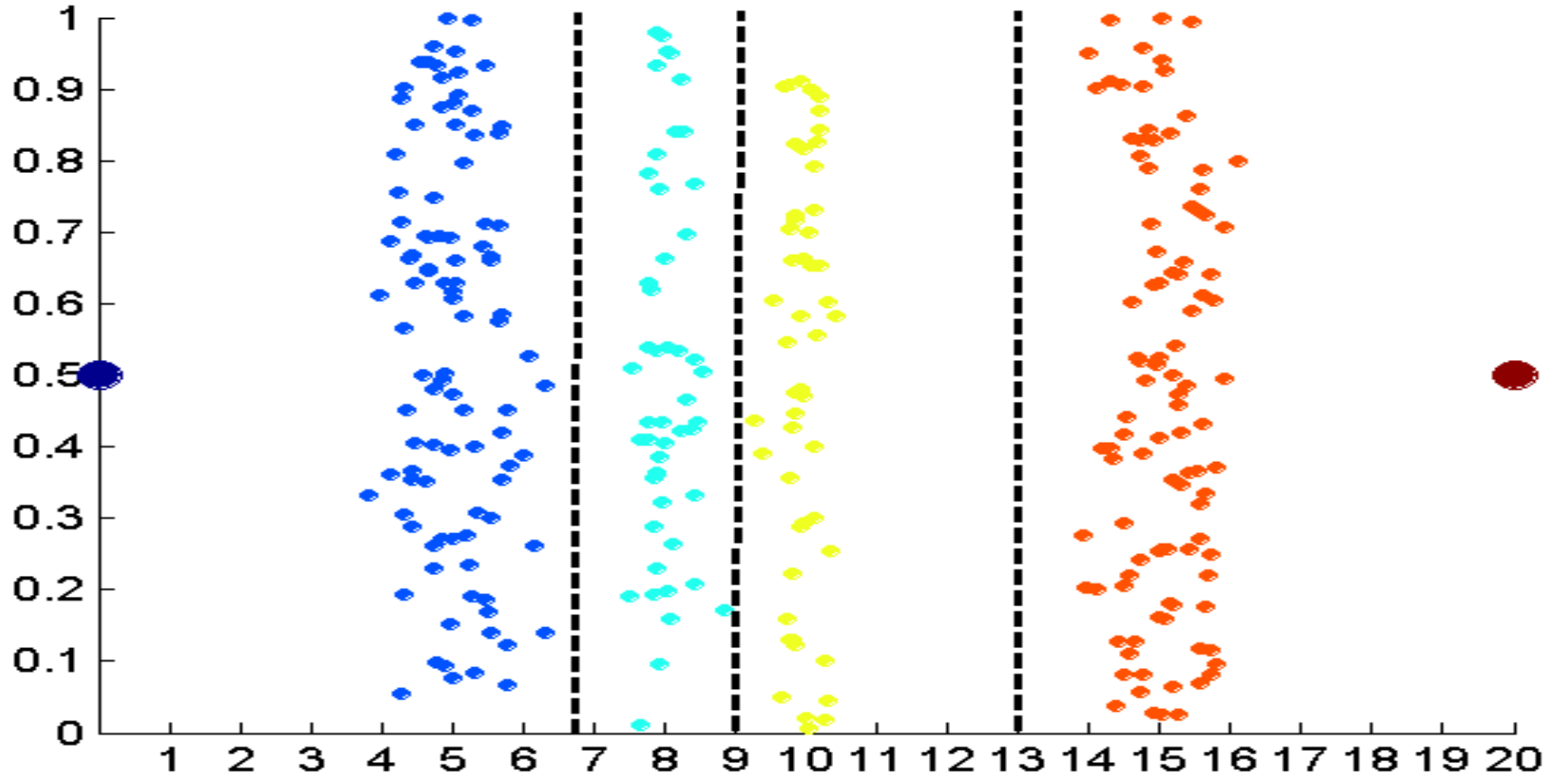
Equal interval width approach used to obtain 4 values.

Unsupervised Discretization



Equal frequency approach used to obtain 4 values.

Unsupervised Discretization



K-means approach to obtain 4 values.

Discretization steps

Any discretization process consists of two basic steps:

- first, the number of discrete intervals needs to be decided. It is usually done by the user, although a few discretization algorithms are able to do it on their own.

- second, the width (boundary) of each interval must be determined, which is often done by a discretization algorithm itself.

Discretization: Number of intervals?

- Number of discretization intervals:

  large number – more of the original information is retained

  small number – the new feature is "easier" for subsequently used learning algorithms

- It should not be smaller than the number of classes one wants to recognize
- The rule of thumb formula: M / (3*C)

  where:

  M – number of training examples

  C – number of classes

Effect of Size of Intervals

- MinSup **If interval size is small => item support can be low**

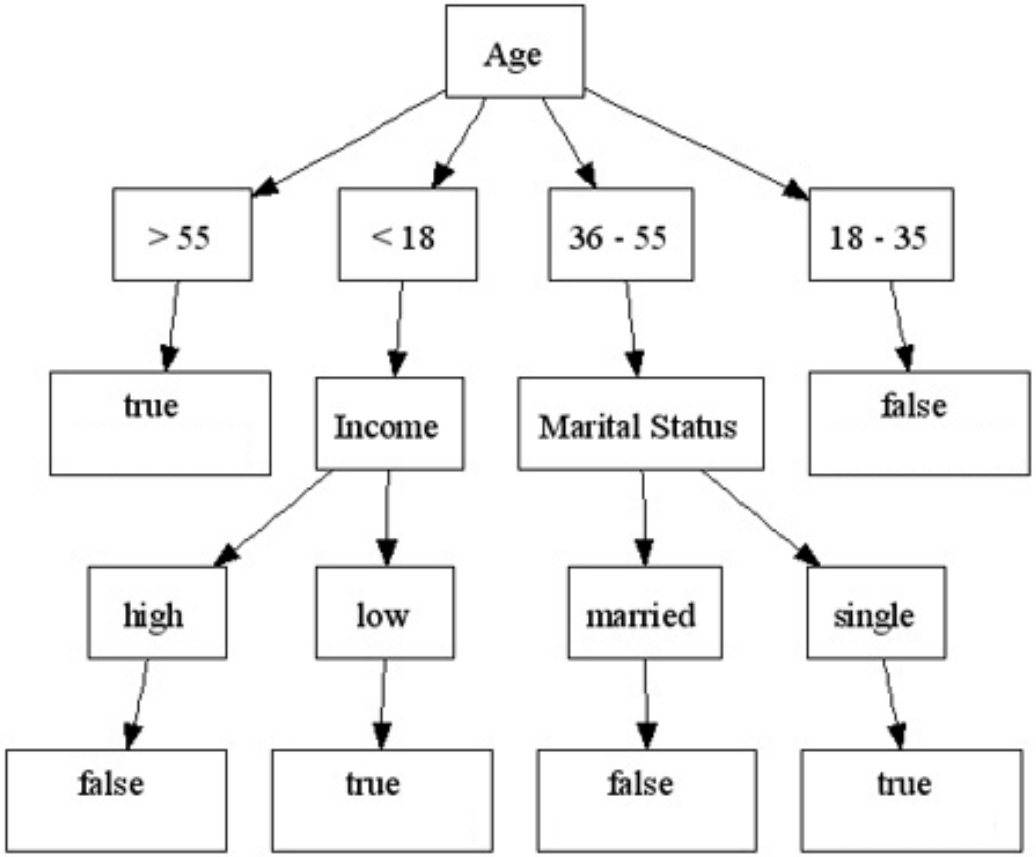
TID	Age	Car
1	21	1
2	29	2
3	35	1
4	38	2

Intervals size = 10

Age	21-30	31-40
Support	2	2

Intervals size = 5

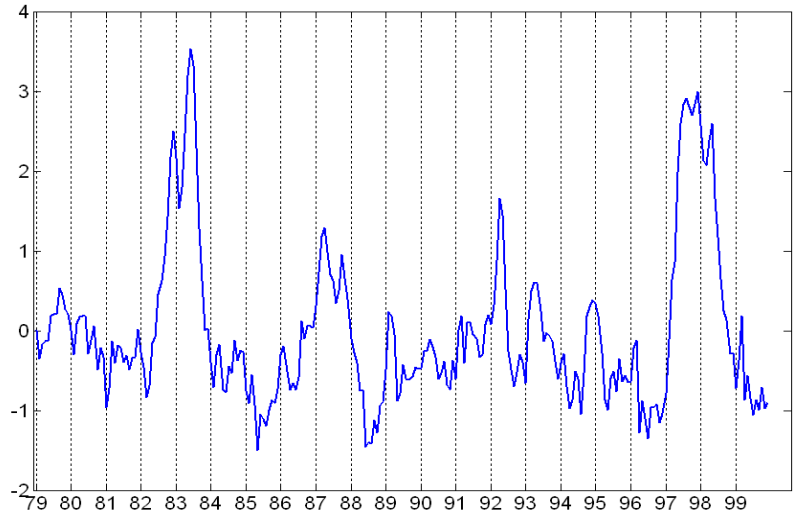
Age	21-25	26-30	31-35	36-40
Support	1	1	1	1



Discrete decision tree

Attribute Transformation

- A function that maps the entire set of values of a given attribute into a new set of replacement values such that each old value can be identified with one of the new values
  - Simple functions: x^k, log(x), e^x, |x|
  - Standardization & Normalization



Feature Scaling: Put Features on same scale

- Features with the largest scales would dominate and skew result
- Some learning techniques require features to be at same scales
  - K-Mean
  - K-Nearest Neighbors
  - Neural Network
  - Linear regression???

- Some learning techniques not require features to be at same scales
  - Decision Trees
  - Random Forrest
  - Xgboost

Effect of different scales in distance calculation

Sample	Age	Salary	NumChild
A	30	50,000	1
B	19	8,000	0
C	45	150,000	2
D	65	35,000	3

Distance (A,B)

$= \cong\sqrt{(30-19)^2 + (50{,}000-8{,}000)^2 + (1-0)^2}$

*** Salary dominates distance between two samples

Feature Scaling

- **Methods**
  - **Normalization**: Scaled to fall within a smaller, specified range
    - **z-score normalization (standardization)**
    - min-max normalization
    - **robust** scaling
    - **Quantile normalization**
  - **Other methods**

Standardization & Normalization (min-max scaling)

- **Puts features on same scale**
- **Standardization (z-score normalization)**:
  - Subtract mean and divide by standard deviation
- **Normalization (min-max scaling)**:
  - Scale to 0-1 by subtracting minimum and dividing by the range

Standardization (Z-score normalization)

- **Normal Distribution** for each feature is assumed

$$x_{i,new} = \frac{x_i - \text{mean}(x)}{\text{stdev}(x)} \qquad x_i \in x$$

- **Not the best** scaling method to use

http://benalexkeen.com/feature-scaling-with-scikit-learn/

Normalization (Min-Max Scaling)

Shrink the range of data into

- [0,1] if all values are positive

$$x_{i,new} = \frac{x_i - \min(x)}{\max(x) - \min(x)} \qquad x_i \in x$$

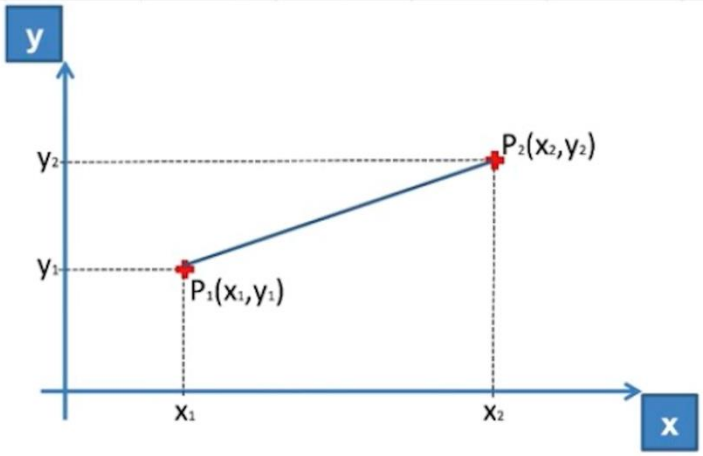
- **[-1,1] if there are negative values**

$$x_{i,new} = \frac{x_i - \min(x)}{\max(x) - \min(x)} * 2 - 1 \qquad x_i \in x$$

http://benalexkeen.com/feature-scaling-with-scikit-learn/

Standardization & Normalization (Min-Max Scaling)

Country	Age	Salary	Purchased
France	44	72000	No
Spain	27	48000	Yes
Germany	30	54000	No
Spain	38	61000	No
Germany	40	63777.77778	Yes
France	35	58000	Yes
Spain	38.77777778	52000	No
France	48	79000	Yes
Germany	50	83000	No
France	37	67000	Yes



Euclidean Distance between P_1 and $P_2 = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$

Standardisation	Normalisation
$x_{\text{stand}} = \frac{x - \text{mean}(x)}{\text{standard deviation } (x)}$	$x_{\text{norm}} = \frac{x - \min(x)}{\max(x) - \min(x)}$

Standardization Example

For variable Age:
Mean=44.66667 and Standard deviation (std) =14.97554

For variable Income ($):
Mean=98.66667 and Standard deviation (std) =62.86706

For normalizing age for observation # 1 we calculate as below. Here age = 25.

$$\frac{25 - mean}{std} = \frac{25 - 44.66667}{14.97554} = -1.31325$$ This can be achieved using Excel's "standardize" function.
Similarly we compute the normalized age and normalized income for all remaining observations in Table 2.3.

Ob. No.	Age	Income ($)	Normalized Age	Normalized Income ($)
1	25	49000	-1.313252974	-0.790026927
2	56	156000	0.756789849	0.911977392
3	65	99000	1.357770024	0.005302194
4	32	192000	-0.845823949	1.48461436
5	41	39000	-0.244843775	-0.949092751
6	49	57000	0.289360825	-0.662774268
Mean	**44.6667**	**98666.67**		
Standard deviation	**14.9755**	**62867.06**		

Standardization with Python

```
from sklearn.preprocessing import StandardScaler

data = [[2, 2000],
        [3.5, 5500],
        [1.75, 2150],
        [3.14, 4130],
        [2.25, 2564]]

scaler = StandardScaler()
scaler.fit(data)
data2 = scaler.transform(data)
print(data2)
print()

data_test = [[1.23, 3210]]
print(scaler.transform(data_test))
```

```
[[-0.78178395 -0.9416242 ]
 [ 1.43919317  1.65585743]
 [-1.1519468  -0.83030356]
 [ 0.90615866  0.63912891]
 [-0.41162109 -0.52305859]]

[[-1.92188553 -0.04363769]]
```

Standardization with Rapidminer

- **z_transformation**: This is also called statistical normalization. This normalization subtracts the mean of the data from all values and then divides them by the standard deviation. Afterwards, the distribution of the data has a mean of zero and a variance of one. This is a common and very useful normalization technique. It preserves the original distribution of the data and is less influenced by outliers.
- **range_transformation**: Range transformation normalizes all Attribute values to a specified value range. When this method is selected, two other parameters (min, max) appear in the Parameters panel. So the largest value is set to 'max' and the smallest value is set to 'min'. All other values are scaled, so they fit into the given range. This method can be influenced by outliers, because the bounds move towards them. On the other hand, this method keeps the original distribution of the data points, so it can also be used for data anonymization, for example to obfuscate the true range of observations.
- **proportion_transformation**: This normalization is based on the proportion each Attribute value has on the complete Attribute. This means each value is divided by the total sum of that Attribute values. The sum is only formed from finite values, ignoring NaN/missing values as well as positive and negative infinity. When this method is selected, another parameter (allow negative values) appears in the Parameters panel. If checked, negative values will be treated as absolute values, otherwise they will produce an error when executed.
- **interquartile_range**: Normalization is performed using the interquartile range. The interquartile range is the distance between the 25th and 75th percentile, which are also called lower and upper quartile, or Q1 and Q3. They are calculated by first sorting the data and then taking the data value that separates the first (or the last) 25% of the Examples from the rest. The median is the 50th percentile, so it is the value that separates the sorted values in half. The interquartile range (IQR) is the difference between Q3 and Q1. The final formula for the interquartile range normalization is then: (value median) / IQR The IQR is the range between the middle 50% of the data, so this normalization method is less influenced by outliers. NaN/missing values, as well as infinite values will be ignored for this method. Also, if no finite values could be found, the corresponding Attribute will be ignored.

Normalization or Standardization?

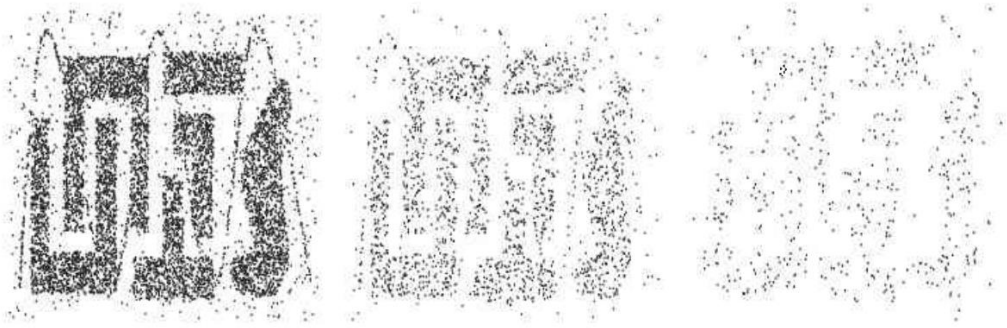
- **Most of the time => Standardization is better**
- **However, Normalization (Min-Max) is faster for**
  - Neural network
  - Linear regression
  - Etc.

Sampling

- Sampling is the main technique employed for data selection.
  - It is often used for both the preliminary investigation of the data and the final data analysis.
  - Can be used to reduce bias
- Statisticians sample because obtaining the entire set of data of interest is too expensive or time-consuming.
- Sampling is used in data analytics because processing the entire set of data of interest is too expensive or time-consuming.

Sampling

- **The key principle for effective sampling is the following:**
  - using a sample will work almost as well as using the entire data sets, if the sample is representative
  - A sample is representative if it has approximately the same property (of interest) as the original set of data
  - Example: Loss of structure with sampling



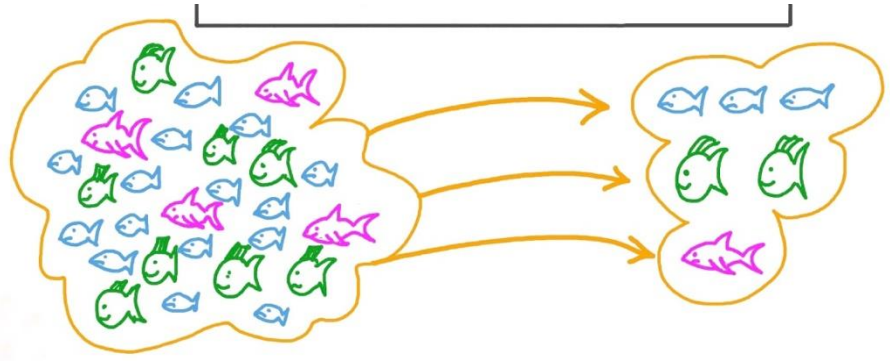
(a) 8000 points (b) 2000 points (c) 500 points

Types of Sampling

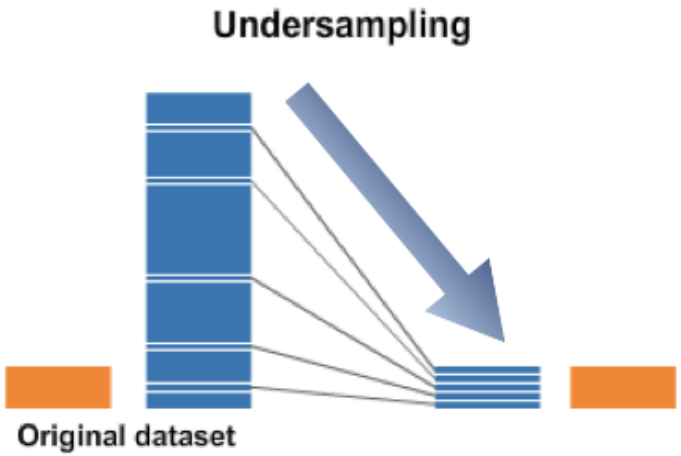
- Simple Random Sampling
  - There is an equal probability of selecting any particular item

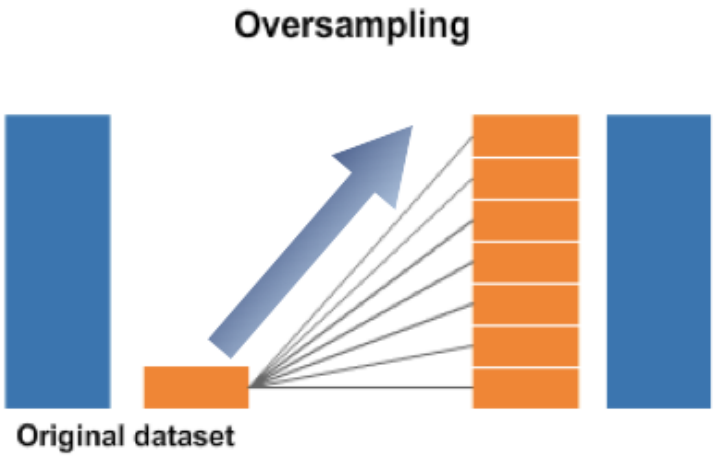
- Stratified sampling
  - Smaller groups which are represented proportionally to the population
  - It ensures that each subgroup is adequately represented in the sample, reducing the potential bias that can arise from uneven distribution of data.
  - Split the data into several partitions; then draw random samples from each partition

- Other types of sampling

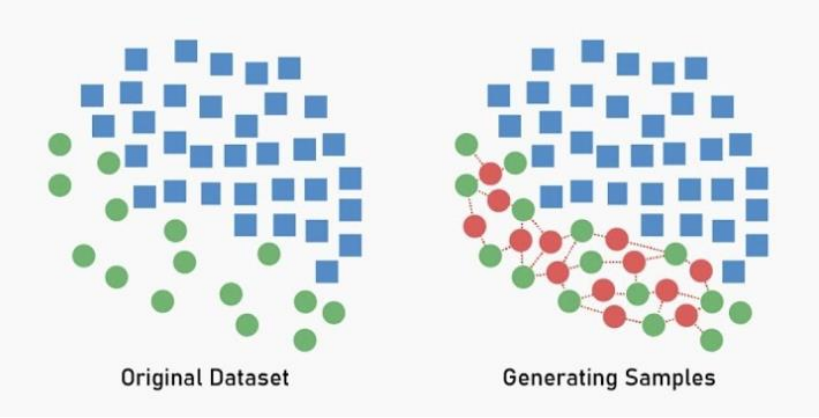


Imbalanced Data => Over-Sampling VS Under-Sampling





- Over-Sampling Method
  - For example, SMOTE
- Under-Sampling Method
  - For example, RUS



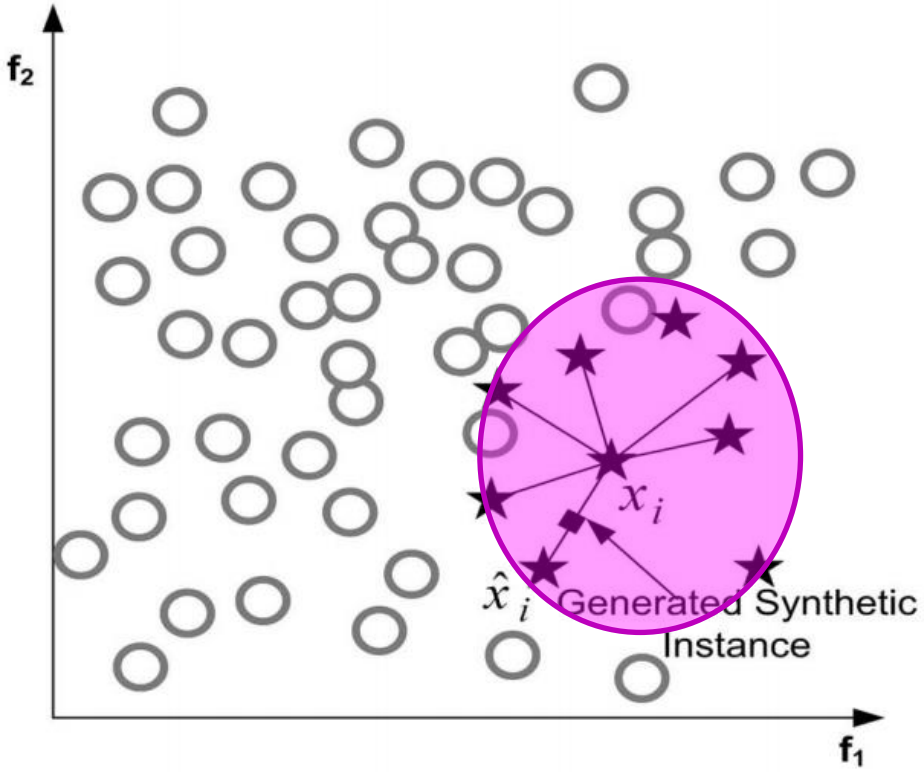
SMOTE: Synthetic over sampling with Data Generation

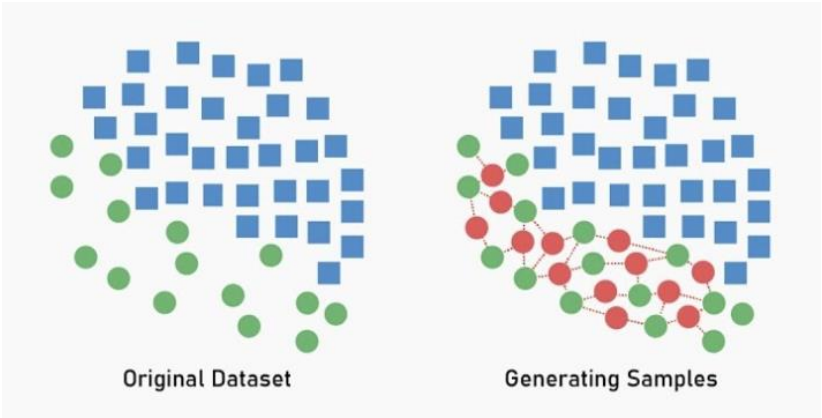
- A powerful method
- Create artificial data by

  Step1: For each minority sample X_i, find its K-nearest neighbors

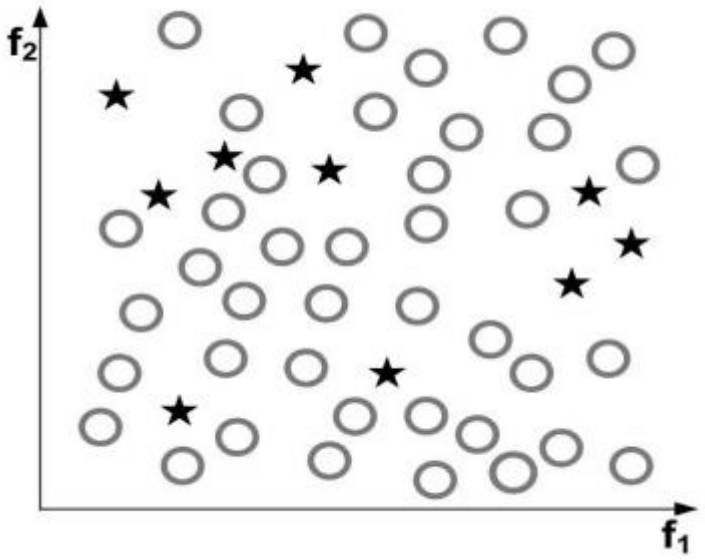
  Step2: Randomly select one of the K-nearest neighbors, $\bar{X}_i$

  Step3: Randomly generate X_{new} on the path form X_i to $\bar{X}_i$

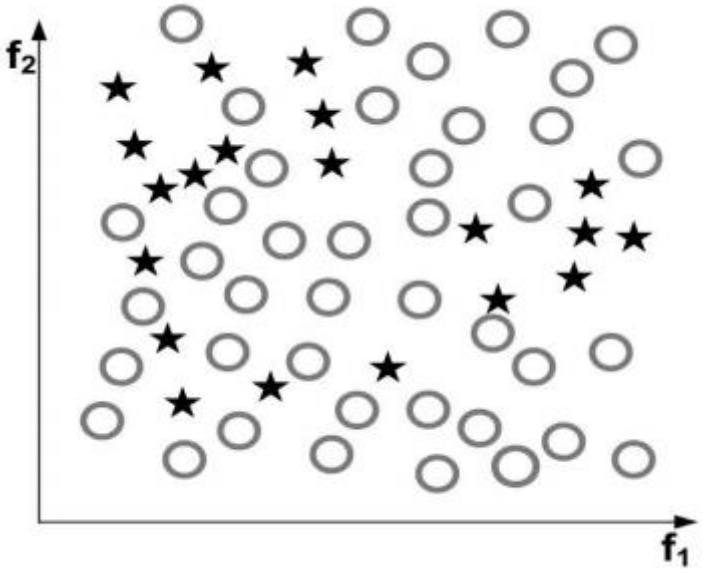




SMOTE Over Sampling Technique



(a) Original data set distribution



(b) Post-SMOTE data set

Aggregation & Generalization

- Combining two or more attributes (or objects)

  => Single attribute (or object)
- Purpose
  - Data reduction
    - Reduce the number of attributes or objects
  - Change of scale
    - Cities aggregated => regions, states, countries, etc
  - More “stable” data
    - Aggregated data tends to have **less variability**

Transaction ID	Item	Store Location	Date	Price	...
⋮	⋮	⋮	⋮	⋮	
101123	Watch	Chicago	09/06/04	$25.99	...
101123	Battery	Chicago	09/06/04	$5.99	...
101124	Shoes	Minneapolis	09/06/04	$75.00	...
⋮	⋮	⋮	⋮	⋮	

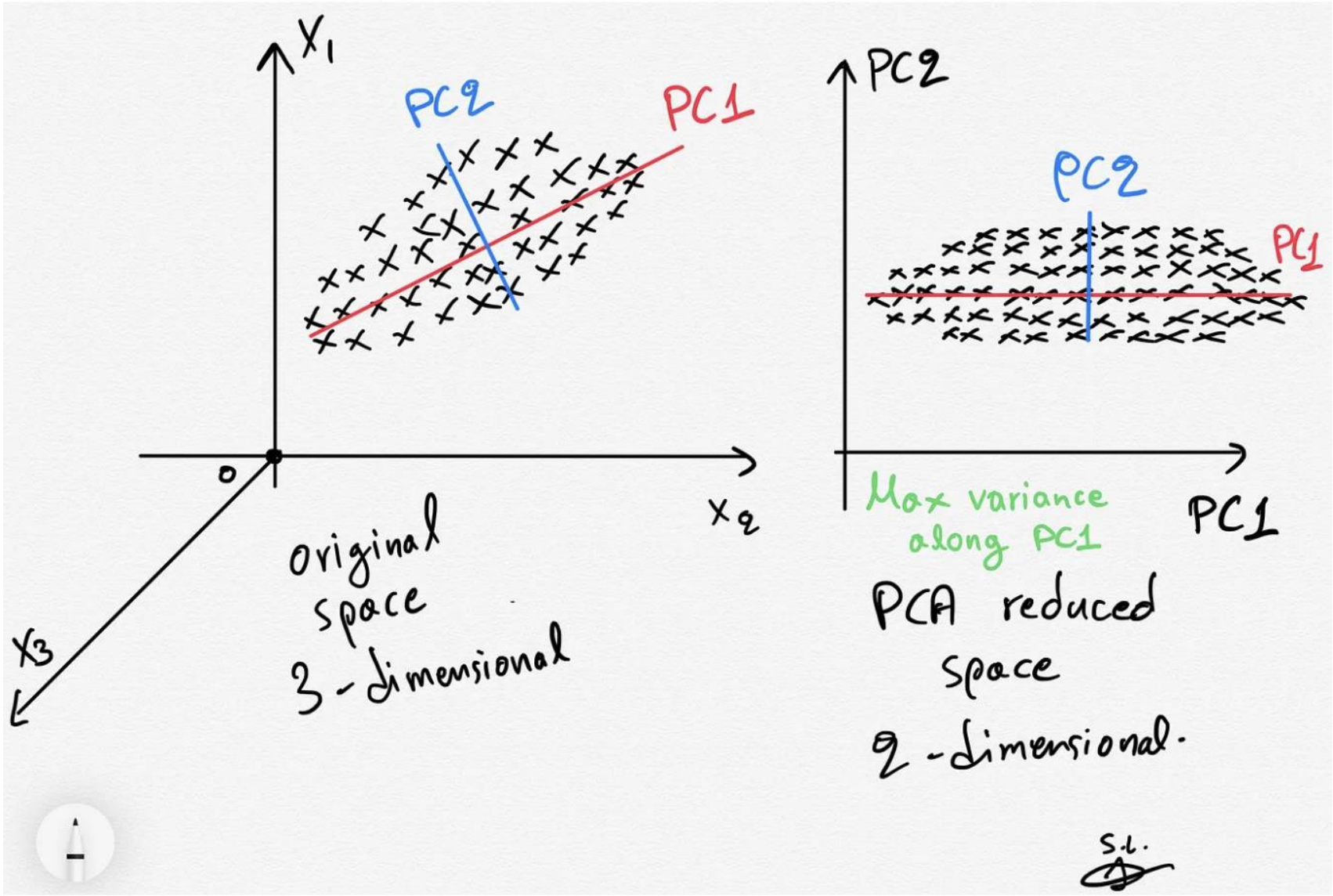
Feature Reduction

- **Purpose**
  - Avoid curse of dimensionality
  - Reduce amount of time and memory required by data mining algorithms
  - Allow data to be more easily visualized
  - May help to eliminate irrelevant features or reduce noise
  - Dimensionality reduction techniques can be categorized into feature extraction & feature selection.

- **Feature extraction** approaches project features into a new feature space with lower dimensionality & the new constructed features are usually combinations of original features
  - Feature extraction techniques
    - Domain specific
    - Principle Component Analysis (PCA)
    - Singular Value Decomposition (SVD)
    - Deep learning,
    - Etc.

- **Feature subset selection** approaches aim to select a small subset of features that minimize redundancy & maximize relevance to the target such as the class labels in classification.

Feature Reduction using PCA



Example – Breakfast Cereals

name	mfr	type	calories	protein	…	rating
100%_Bran	N	C	70	4	…	68
100%_Natural_Bran	Q	C	120	3	…	34
All-Bran	K	C	70	4	…	59
All-Bran_with_Extra_Fiber	K	C	50	4	…	94
Almond_Delight	R	C	110	2	…	34
Apple_Cinnamon_Cheerios	G	C	110	2	…	30
Apple_Jacks	K	C	110	2	…	33
Basic_4	G	C	130	3	…	37
Bran_Chex	R	C	90	2	…	49
Bran_Flakes	P	C	90	3	…	53
Cap'n'Crunch	Q	C	120	1	…	18
Cheerios	G	C	110	6	…	51
Cinnamon_Toast_Crunch	G	C	120	1	…	20

Example – Breakfast Cereals Description of Variables

Name: name of cereal

mfr: manufacturer

type: cold or hot

calories: calories per serving

protein: grams

fat: grams

sodium: mg.

fiber: grams

carbo: grams complex carbohydrates

sugars: grams

potass: mg.

vitamins: % FDA rec

shelf: display shelf

weight: oz. 1 serving

cups: in one serving

rating: consumer reports

PCA Output

Variable	1	2	3	4	5	6
calories	0.07624155	-0.01066097	0.61074823	-0.61706442	0.45754826	0.12601775
protein	-0.00146212	0.00873588	0.00050506	0.0019389	0.05533375	0.10379469
fat	-0.00013779	0.00271266	0.01596125	-0.02595884	-0.01839438	-0.12500292
sodium	0.98165619	0.12513085	-0.14073193	-0.00293341	0.01588042	0.02245871
fiber	-0.00479783	0.03077993	-0.01684542	0.02145976	0.00872434	0.271184
carbo	0.01486445	-0.01731863	0.01272501	0.02175146	0.35580006	-0.56089228
sugars	0.00398314	-0.00013545	0.09870714	-0.11555841	-0.29906386	0.62323487
potass	-0.119053	0.98861349	0.03619435	-0.042696	-0.04644227	-0.05091622
vitamins	0.10149482	0.01598651	0.7074821	0.69835609	-0.02556211	0.01341988
shelf	-0.00093911	0.00443601	0.01267395	0.00574066	-0.00823057	-0.05412053
weight	0.0005016	0.00098829	0.00369807	-0.0026621	0.00318591	0.00817035
cups	0.00047302	-0.00160279	0.00060208	0.00095916	0.00280366	-0.01087413
rating	-0.07615706	0.07254035	-0.30776858	0.33866307	0.75365263	0.41805118

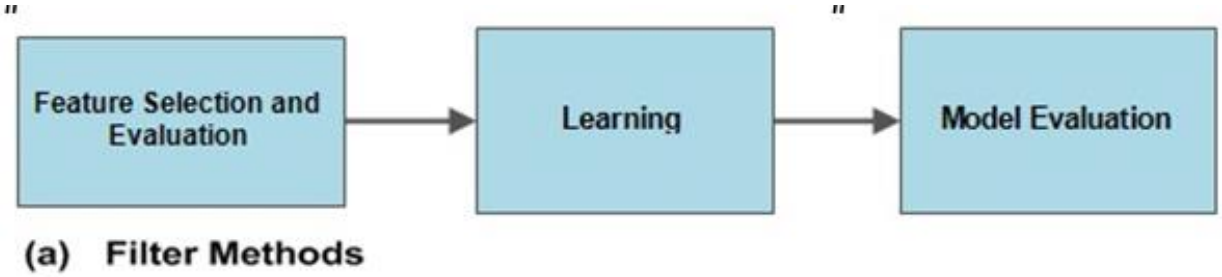
Variance	7204.161133	4833.050293	498.4260864	357.2174377	72.47863007	4.33980322
Variance%	55.52834702	37.25226212	3.84177661	2.75336623	0.55865192	0.0334504
Cum%	55.52834702	92.78060913	96.62238312	99.37575531	99.93440247	99.96785736

- First 6 components shown
- First 2 capture 93% of the total variation

Feature Subset Selection

- How can we find a optimal(good, minimal) subset of the original attributes
- **Redundant features** duplicate much or all of the information contained in one or more other attributes
  - Example: Constant Features (Variance = 0) must be deleted !
- **Irrelevant features** contain no information that is useful for the data mining task at hand
  - Example: students' ID is often irrelevant to the task of predicting students' GPA
- Feature selection aims to select subset of highly discriminant features.
- It selects features that are capable of **discriminating samples that belong to different classes**.
- Feature selection for classification attempts to select the minimally sized subset of features according to the following criteria
  - the classification accuracy does not significantly decrease; and
  - the resulting class distribution is as close the original class distribution
- Brute-force approach:
  - Try all possible feature subsets as input to data mining algorithm

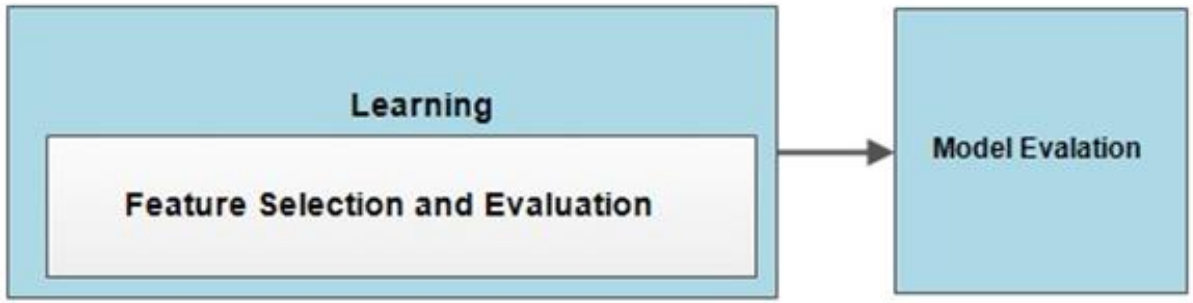
Feature Subset Selection



(a) Filter Methods



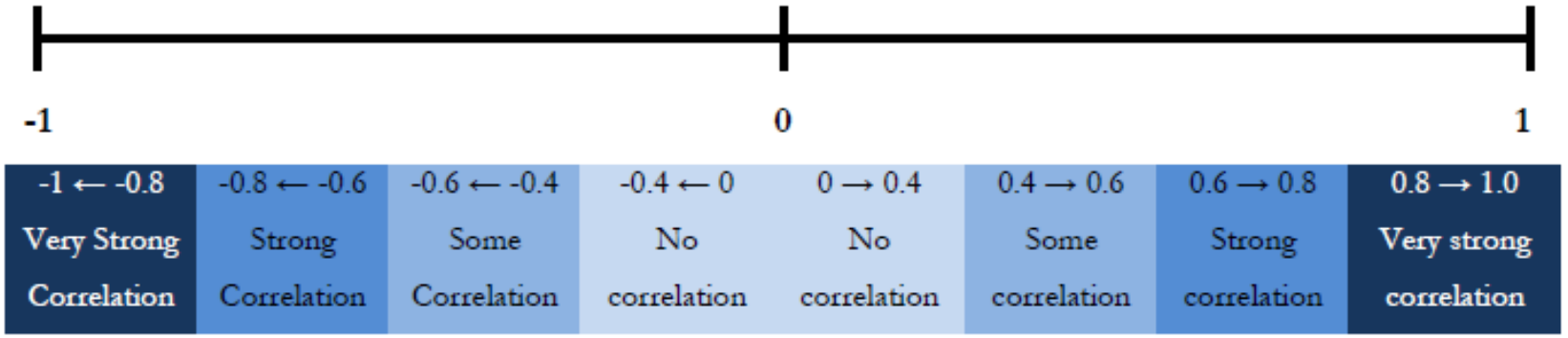
(b) Wrapper Methods



(c) Embedded Methods

Filter approach for feature selection: Correlation

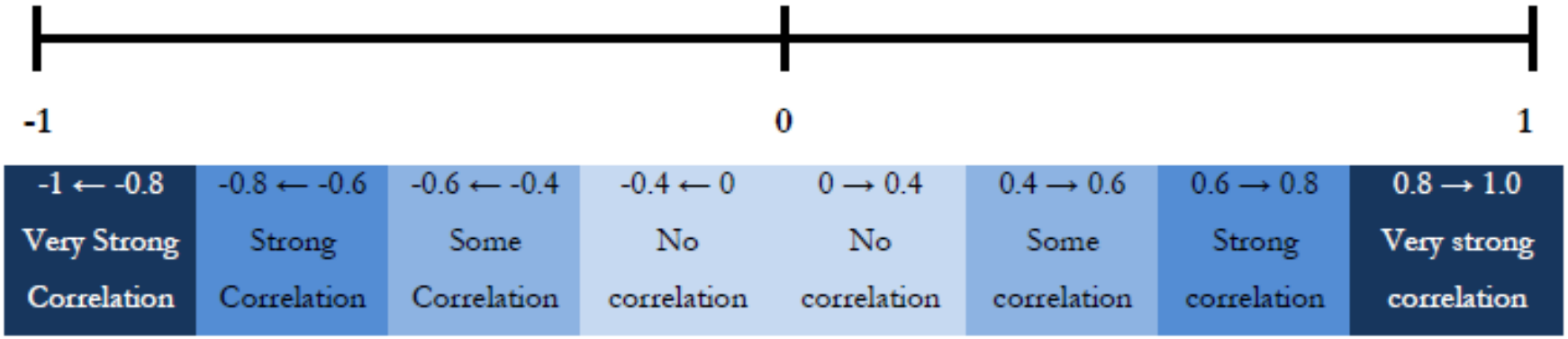
- All correlation coefficients between 0 and 1 represent positive correlations, while all coefficients between 0 and -1 are negative correlations



Correlation

Pearson Correlation Coefficient (r):

- r = 0 no correlation
- r = -1 perfect negative linear correlation
- r = 1 perfect positive linear correlation



Correlation

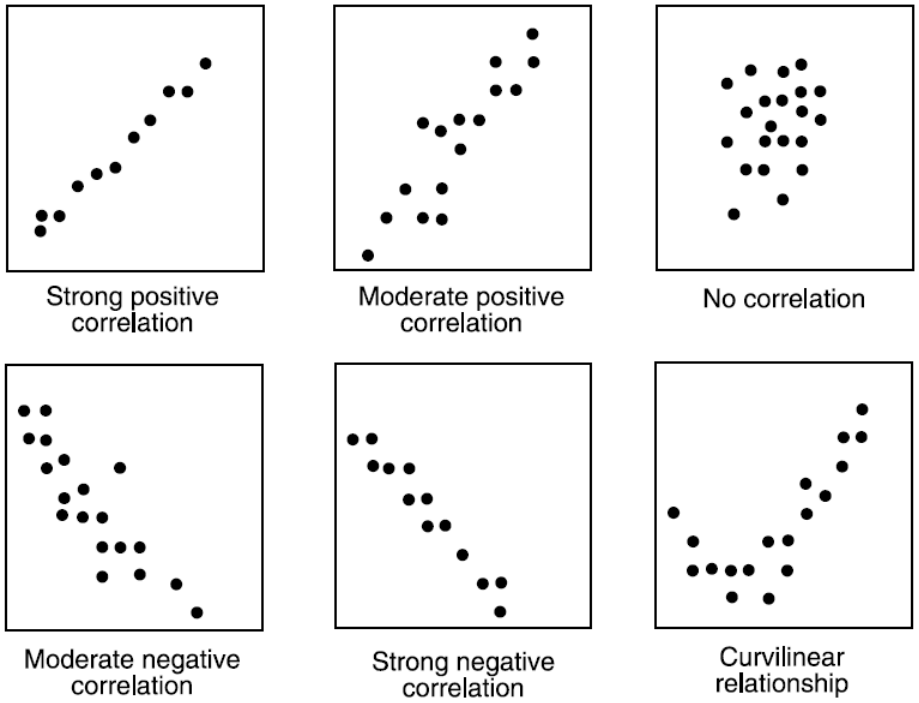
Correlation

When two variables move together,
we say they are *correlated*.

Correlation

Scatter Plot

- Show relationship between two variables
- Not indicates the strength of relationship amongst them.



Filter approach for feature selection: Correlation

Attributes	Model	Price	Age_08_04	Mfg_Month	Mfg_Year	KM	Fuel_Type
Model	1	-0.647	0.732	-0.010	-0.735	0.313	0.103
Price	-0.647	1	-0.877	-0.018	0.885	-0.570	-0.053
Age_08_04	0.732	-0.877	1	-0.123	-0.984	0.506	0.098
Mfg_Month	-0.010	-0.018	-0.123	1	-0.057	-0.021	-0.052
Mfg_Year	-0.735	0.885	-0.984	-0.057	1	-0.505	-0.089
KM	0.313	-0.570	0.506	-0.021	-0.505	1	-0.417
Fuel_Type	0.103	-0.053	0.098	-0.052	-0.089	-0.417	1

- **Price and age are highly negatively correlated**
  - **it is reasonable that older cars are cheaper**
- **Price and Kilometers also appear negatively correlated**
  - **cars with higher mileage are cheaper**

Correlation

Pearson product moment Correlation coefficient (r):

$$r = \frac{\text{Covariance}(X,Y)}{\text{SQRT}(\text{Variance}(X)*\text{Variance}(Y))}$$

$$\text{Covariance}(X,Y) = \frac{\Sigma(X-\overline{X})(Y-\overline{Y})}{n-1}$$

$$\text{Variance}(X) = \frac{\Sigma(X-\overline{X})^2}{n-1} \qquad \text{Variance}(Y) = \frac{\Sigma(Y-\overline{Y})^2}{n-1}$$

Correlation

Problem definition:

To predict, whether the students will play cricket or not?

Variables:

Categorical: Gender, PlayCricket

Continuous: Prev_Exam_Marks, Height (cm), Weight (Kgs)

Student_ID	Gender	Prev_Exam_Marks	Height (cm)	Weight Caregory (kgs)	Play Cricket
S001	M	65	178	61	1
S002	F	75	174	56	0
S003	M	45	163	62	1
S004	M	57	175	70	0
S005	F	59	162	67	0

Correlation

Find out the Pearson product moment correlation coefficient (r) of Prev_Exam_Marks & Weight.

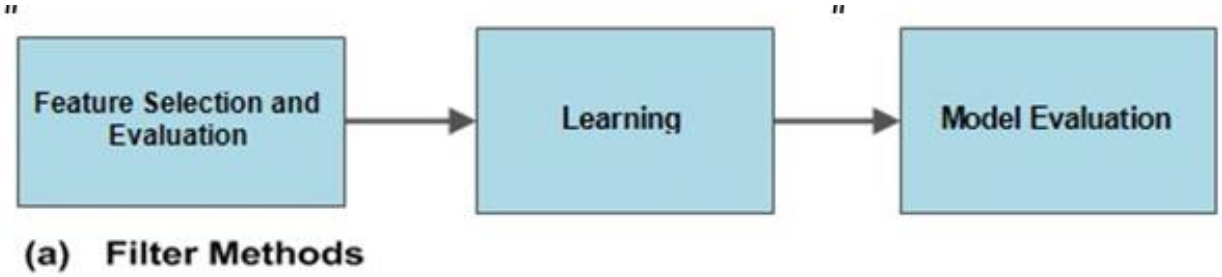
- Mean of Prev_Exam_Marks and Weight
- Covariance(Prev_Exam_Marks, Weight)
- Variance(Prev_Exam_Marks), Variance(Weight)

Student_ID	Prev_Ex_Marks: x	$x-\bar{x}$	$(x-\bar{x})^2$	Weight (kgs): y	$y-\bar{y}$	$(y-\bar{y})^2$	$(x-\bar{x})(y-\bar{y})$
S001	65			65			
S002	75			75			
S003	45			45			
S004	57			57			
S005	59			59			
	sum			sum			

Correlation VS Causation

- Correlation: When two variables move together,
  - We say they are correlated.
- Causation: When changes in one variable (x) affect changes in another variable (y), we say that X causes Y.
  - Examples: Education -> Higher wages
- Correlation does not necessarily imply causality
  - because correlation measures the statistical relationship between two variables, indicating how they are related or vary together.
- Correlation shows the strength and direction of the relationship but does not provide information about the cause-and-effect relationship between them.

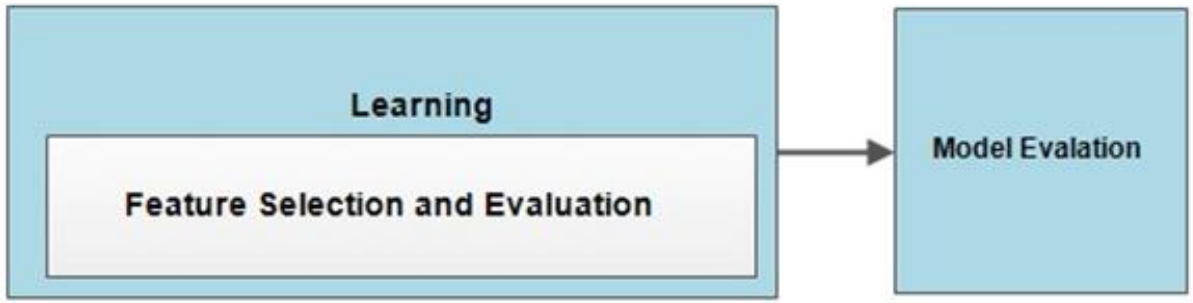
Feature Subset Selection



(a) Filter Methods



(b) Wrapper Methods



(c) Embedded Methods

Filter approach for feature selection: Impurity Measures

- Impurity Measures
  - Gini Index
  - Entropy
  - Information-Gain
  - Misclassification error
  - Chi-square
  - Etc.

Purity VS Impurity in Data

Need a measure of degree of impurity:

C0: 9 C1: 1

Homogeneous (Pure),

Low degree of impurity

Class
C0
C0
C0
C0
C0
C0
C0
C0
C0
C1

C0: 5 C1: 5

Non-homogeneous,

High degree of impurity

Class
C0
C0
C0
C0
C0
C1
C1
C1
C1
C1

Filter approach for feature selection: Impurity Measures

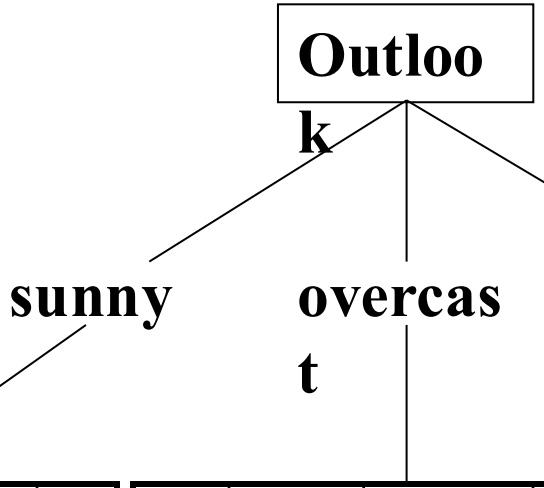
Outlook	Temperature	Humidity	Windy	Class
sunny	hot	high	false	N
sunny	hot	high	true	N
overcast	hot	high	false	P
rain	mild	high	false	P
rain	cool	normal	false	P
rain	cool	normal	true	N
overcast	cool	normal	true	P
sunny	mild	high	false	N
sunny	cool	normal	false	P
rain	mild	normal	false	P
sunny	mild	normal	true	P
overcast	mild	high	true	P
overcast	hot	normal	false	P
rain	mild	high	true	N

Which attribute allows low degree of impurity?

Filter approach for feature selection:

Impurity Measures

Outlook	Temperature	Humidity	Windy	Class
sunny	hot	high	false	N
sunny	hot	high	true	N
overcast	hot	high	false	P
rain	mild	high	false	P
rain	cool	normal	false	P
rain	cool	normal	true	N
overcast	cool	normal	true	P
sunny	mild	high	false	N
sunny	cool	normal	false	P
rain	mild	normal	false	P
sunny	mild	normal	true	P
overcast	mild	high	true	P
overcast	hot	normal	false	P
rain	mild	high	true	N



1	Hot	High	False	N
2	Hot	High	True	N
8	Mild	High	False	N
9	Cool	Normal	False	P
11	Mild	Normal	True	P

3	Hot	High	False	P
7	Cool	Normal	true	P
12	Mild	High	True	P
13	Hot	Normal	False	P

4	Mild	High	False	P
5	Cool	Normal	False	P
6	Cool	Normal	True	N
10	Mild	Normal	False	P
14	Mild	High	True	N

Filter approach for feature selection: Info-Gain

Gain(Outlook) = 0.246
Gain(Temperature) = 0.029
Gain(Humidity) = 0.151
Gain(Windy) = 0.048

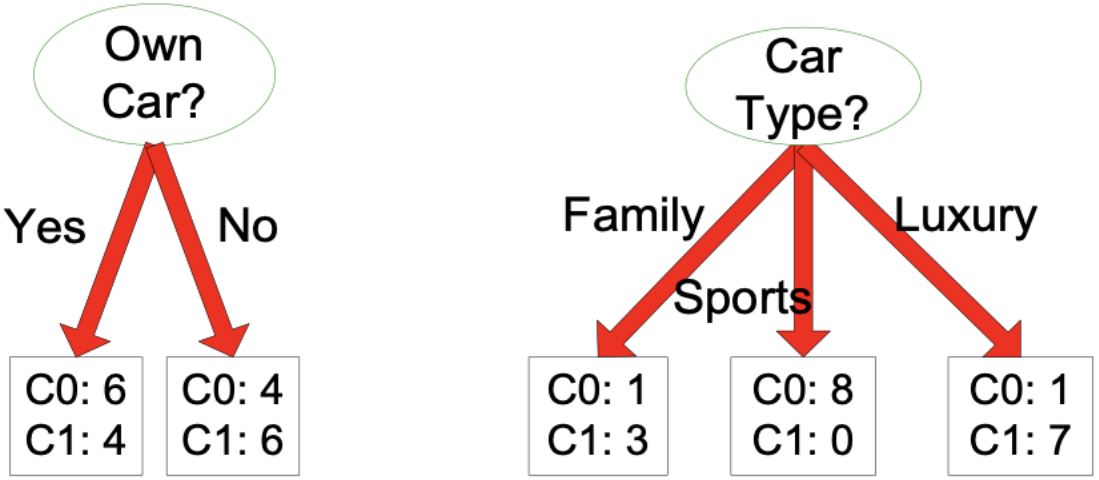
Outlook	Temperature	Humidity	Windy	Class
sunny	hot	high	false	N
sunny	hot	high	true	N
overcast	hot	high	false	P
rain	mild	high	false	P
rain	cool	normal	false	P
rain	cool	normal	true	N
overcast	cool	normal	true	P
sunny	mild	high	false	N
sunny	cool	normal	false	P
rain	mild	normal	false	P
sunny	mild	normal	true	P
overcast	mild	high	true	P
overcast	hot	normal	false	P
rain	mild	high	true	N

Filter approach for feature selection: GINI Index

attribute	weight
Temperature	0.029
Windy	0.048
Humidity	0.152
Outlook	0.247

Outlook	Temperature	Humidity	Windy	Class
sunny	hot	high	false	N
sunny	hot	high	true	N
overcast	hot	high	false	P
rain	mild	high	false	P
rain	cool	normal	false	P
rain	cool	normal	true	N
overcast	cool	normal	true	P
sunny	mild	high	false	N
sunny	cool	normal	false	P
rain	mild	normal	false	P
sunny	mild	normal	true	P
overcast	mild	high	true	P
overcast	hot	normal	false	P
rain	mild	high	true	N

Filter approach for feature selection: Impurity Measure



Which attribute allows low degree of impurity?

Measure of Impurity: GINI

- Gini Index for a given dataset t :

$$GINI(t) = 1 - \sum_j [p(j \mid t)]^2$$

(NOTE: *p(j | t)* is the relative frequency of class j at dataset t).

- Maximum when records are equally distributed among all classes, implying least interesting information
- Minimum when all records belong to one class, implying most interesting information

C1	**0**
C2	**6**
Gini=0.000	

C1	**1**
C2	**5**
Gini=0.278	

C1	**2**
C2	**4**
Gini=0.444	

C1	**3**
C2	**3**
Gini=0.500	

Examples for computing GINI

$$GINI(t) = 1 - \sum_j [p(j \mid t)]^2$$

C1	**0**
C2	**6**

P(C1) = 0/6 = 0 P(C2) = 6/6 = 1

Gini = 1 – $P(C1)^2$ – $P(C2)^2$ = 1 – 0 – 1 = 0

C1	**1**
C2	**5**

P(C1) = 1/6 P(C2) = 5/6

Gini = 1 – $(1/6)^2$ – $(5/6)^2$ = 0.278

C1	**2**
C2	**4**

P(C1) = 2/6 P(C2) = 4/6

Gini = 1 – $(2/6)^2$ – $(4/6)^2$ = 0.444

Highest GINI when equal distribution among all classes => 1-(1/N) where N is the number of classes.

Measure of Impurity: Entropy

- Entropy at a given dataset t:

$$Entropy(t) = -\sum_j p(j \mid t) \log p(j \mid t)$$

(NOTE: *p(j | t)* is the relative frequency of class j at dataset t).

- Measures homogeneity of a dataset.
  - Maximum (log n_c) when records are equally distributed among all classes implying least information
  - Minimum (0.0) when all records belong to one class, implying most information
- Entropy based computations are similar to the GINI index computations

Examples for computing Entropy

$$Entropy(t) = -\sum_j p(j \mid t) \log_2 p(j \mid t)$$

C1	0
C2	6

P(C1) = 0/6 = 0 P(C2) = 6/6 = 1

Entropy = – 0 log 0 – 1 log 1 = – 0 – 0 = 0

C1	1
C2	5

P(C1) = 1/6 P(C2) = 5/6

Entropy = – (1/6) $\log_2$ (1/6) – (5/6) $\log_2$ (5/6) = 0.65

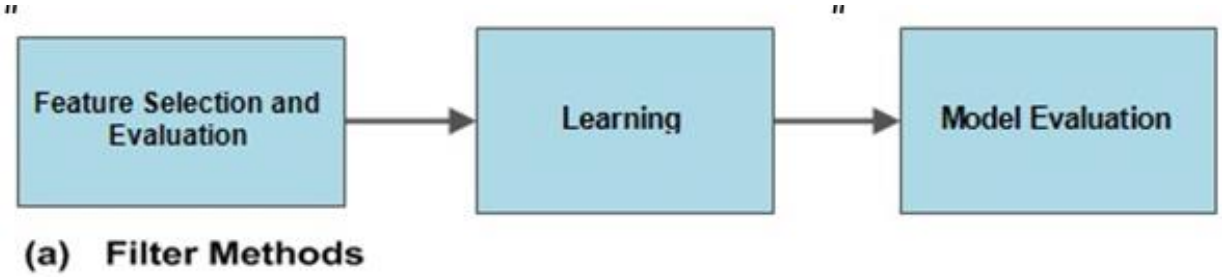
C1	2
C2	4

P(C1) = 2/6 P(C2) = 4/6

Entropy = – (2/6) $\log_2$ (2/6) – (4/6) $\log_2$ (4/6) = 0.92

Highest Entropy when equal distribution among all classes

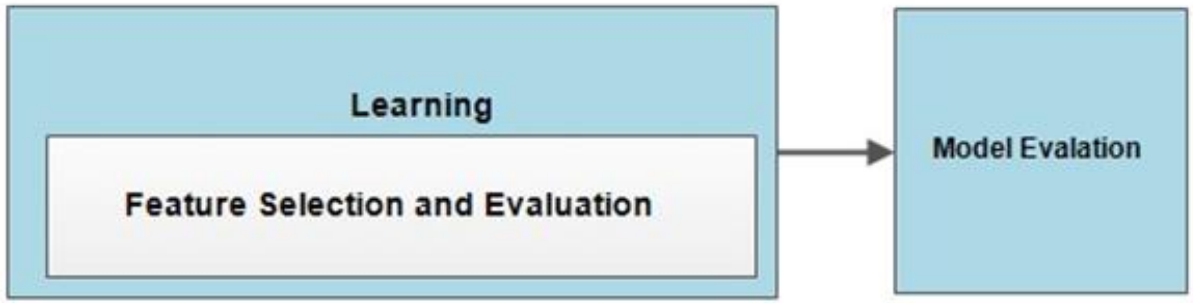
Feature Subset Selection



(a) Filter Methods



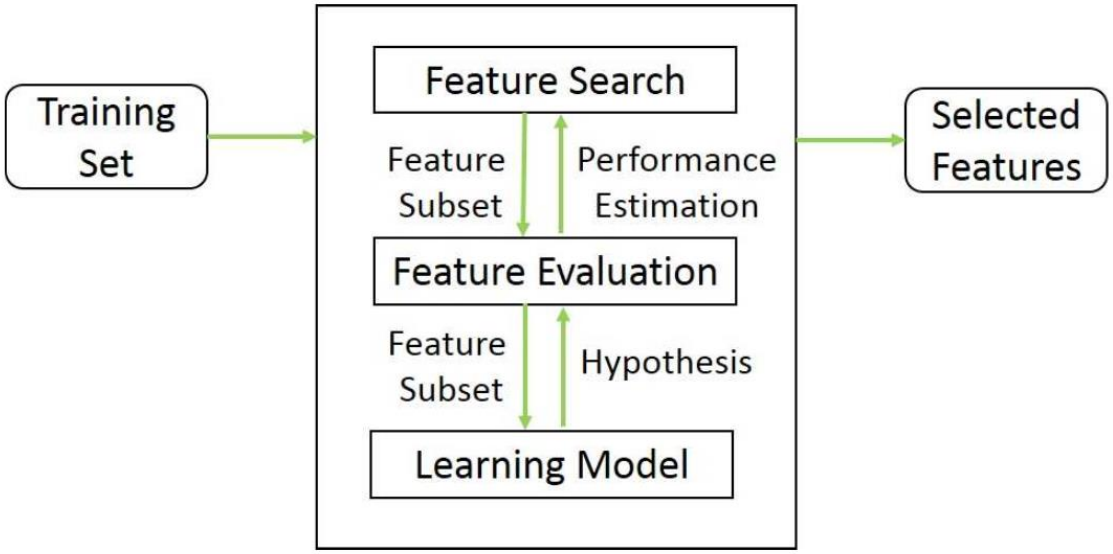
(b) Wrapper Methods



(c) Embedded Methods

Wrapper approach for feature selection

- Wrapper models use a specific learning algorithm to evaluate the quality of the selected features.



- For example, multiple linear regression, etc.

Wrapper approach for feature selection

Selecting Subsets of Predictors

Goal: Find the simplest model that performs sufficiently well

- More robust
- Higher predictive accuracy

Exhaustive Search

Partial Search Algorithms

Wrapper approach for feature selection

Backward/Forward Selection

- **Backward Selection**
  - Start with all predictors
  - Delete them one by one
  - Stop when the addition is not statistically significant

- **Forward Selection**
  - Start with no predictors
  - Add them one by one (add the one with largest contribution)
  - Stop when the addition is not statistically significant

Wrapper approach for feature selection

Diagnostics for the 12 models

	#Coeffs	RSS	Cp	R-Squared	Adj. R-Squared
Choose Subset	2	2538203648	566.4946289	0.759902259	0.759623076
Choose Subset	3	2245803264	404.393219	0.787561455	0.787066837
Choose Subset	4	1796573056	154.2755432	0.830055744	0.829461533
Choose Subset	5	1689283456	96.06230164	0.84020465	0.839458814
Choose Subset	6	1555462272	22.9589653	0.852863273	0.85200383
Choose Subset	7	1516825984	3.27544785	0.856518017	0.855511126
Choose Subset	8	1515638144	4.60880661	0.856630379	0.855455219
Choose Subset	9	1515206272	6.36643076	0.856671232	0.855326999
Choose Subset	10	1514873088	8.1794405	0.856702749	0.855189045
Choose Subset	11	1514592768	10.02211857	0.856729265	0.855045708
Choose Subset	12	1514553344	11.99999332	0.856732995	0.854878951

Pre-processing for supervised learning

- **Hold-out** divides the dataset into two parts.
  - First part is larger and is used as a learning dataset

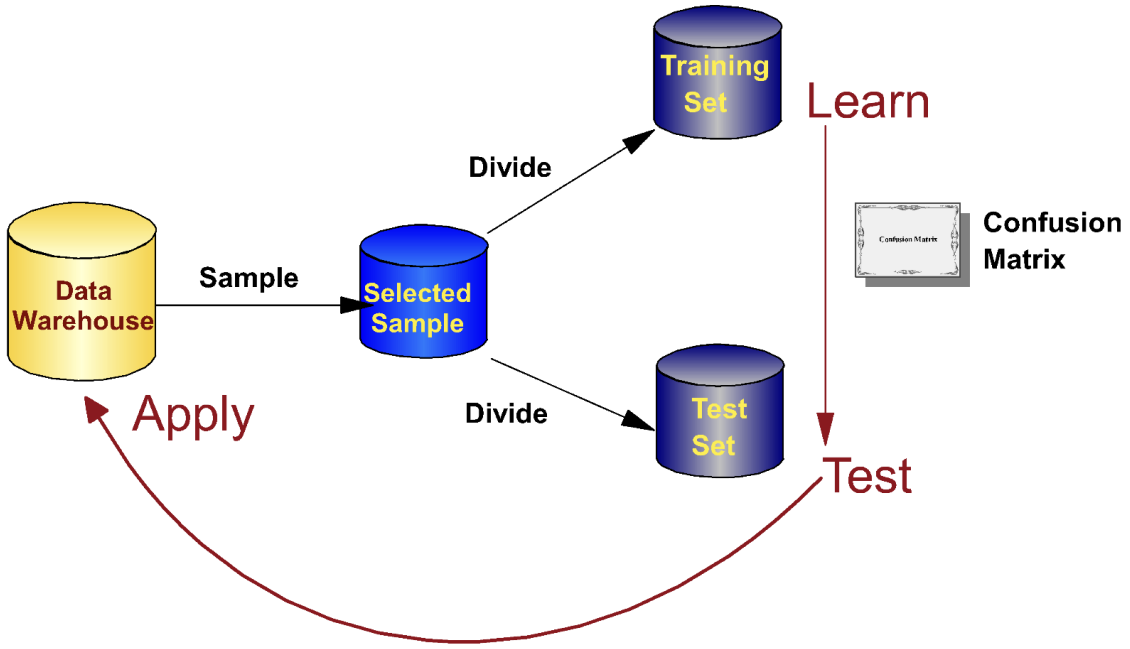
    => Training-Set
  - Second part is test data set to measure the performance of the model.

    => Test-Set
  - Hold-out is used when there is a large amount of data.
  - Therefore, the characteristics and distribution of Training and Test Sets are similar.

- **Other methods**

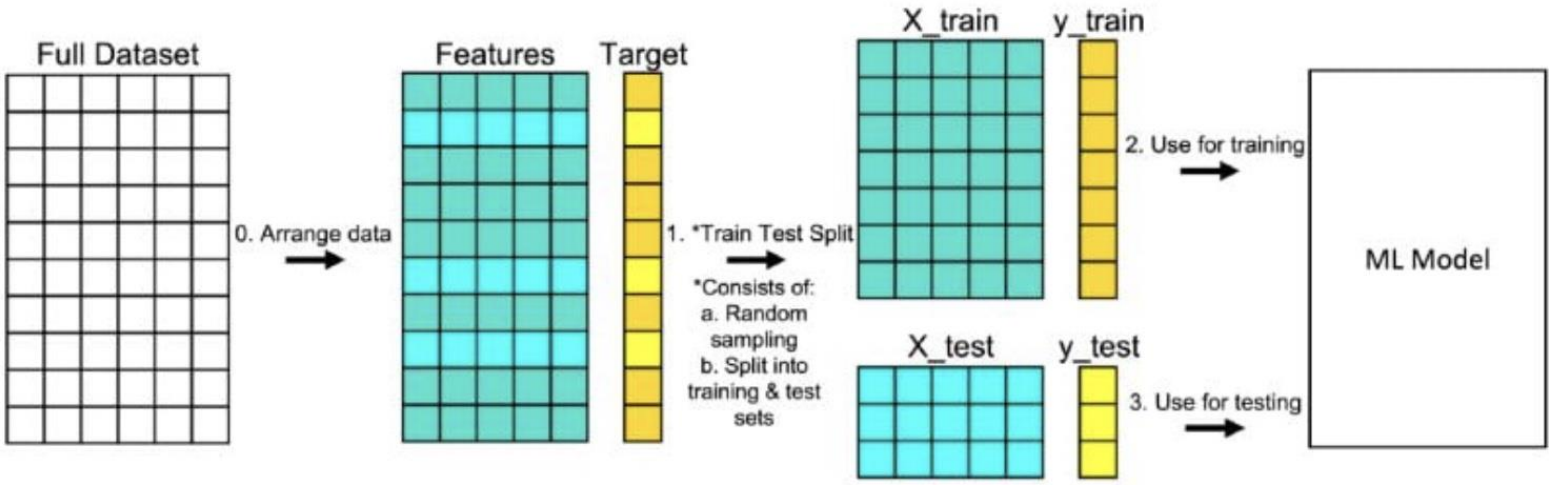
Hold-out divides the dataset into two parts

- Usually, the given data set is divided into training and test set, with training set used to build the model and test set used to validate it.
- A *test set* is used to determine the accuracy of the model.



Python train_test_split

Splitting the dataset into the Training set and Test set



Simple Dataset

Splitting the dataset into the Training set (80%) and Test set (20%)

Row No.	Age	Salary	ID	Country	Purchased
1	44	72000	1	France	No
2	27	48000	2	Spain	Yes
3	30	54000	3	Germany	No
4	38	61000	4	Spain	No
5	40	63778	5	Germany	Yes
6	35	58000	6	France	Yes
7	39	52000	7	Spain	No
8	48	79000	8	France	Yes
9	50	83000	9	Germany	No
10	37	67000	10	France	Yes

ID	Purchased	Country = France	Country = Spain	Country = Germany	Age	Salary
1	No	1	0	0	44	72000
2	Yes	0	1	0	27	48000
3	No	0	0	1	30	54000
4	No	0	1	0	38	61000
5	Yes	0	0	1	40	63778
6	Yes	1	0	0	35	58000
7	No	0	1	0	39	52000
8	Yes	1	0	0	48	79000
9	No	0	0	1	50	83000
10	Yes	1	0	0	37	67000

Python **train_test_split** operator

Splitting the dataset into the Training set (80%) and Test set (20%)

```
[ ] from sklearn.model_selection import train_test_split
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.2, random_state = 1)
```

```
[ ] print(X_train)
```

```
[[0.0 0.0 1.0 38.77777777777778 52000.0]
 [0.0 1.0 0.0 40.0 63777.77777777778]
 [1.0 0.0 0.0 44.0 72000.0]
 [0.0 0.0 1.0 38.0 61000.0]
 [0.0 0.0 1.0 27.0 48000.0]
 [1.0 0.0 0.0 48.0 79000.0]
 [0.0 1.0 0.0 50.0 83000.0]
 [1.0 0.0 0.0 35.0 58000.0]]
```

```
[ ] print(X_test)
```

```
[[0.0 1.0 0.0 30.0 54000.0]
 [1.0 0.0 0.0 37.0 67000.0]]
```

```
[ ] print(y_train)
```

```
[0 1 0 0 1 1 0 1]
```

```
[ ] print(y_test)
```

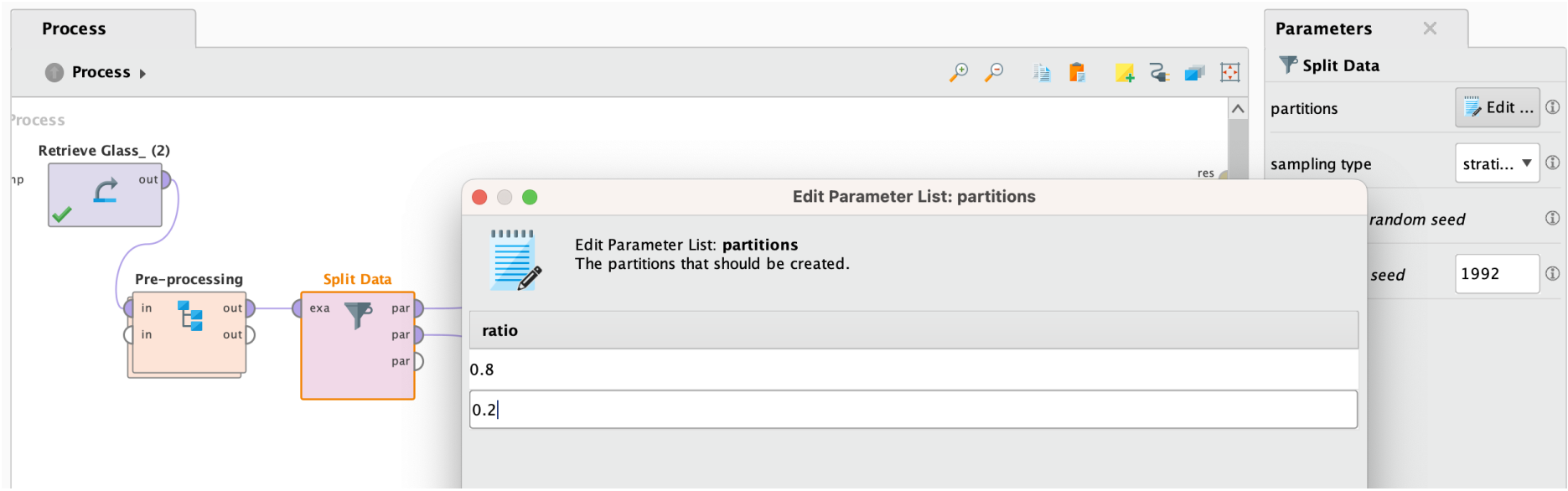
```
[0 1]
```

```
[ ] print(X)
```

```
[[1.0 0.0 0.0 44.0 72000.0]
 [0.0 0.0 1.0 27.0 48000.0]
 [0.0 1.0 0.0 30.0 54000.0]
 [0.0 0.0 1.0 38.0 61000.0]
 [0.0 1.0 0.0 40.0 63777.77777777778]
 [1.0 0.0 0.0 35.0 58000.0]
 [0.0 0.0 1.0 38.77777777777778 52000.0]
 [1.0 0.0 0.0 48.0 79000.0]
 [0.0 1.0 0.0 50.0 83000.0]
 [1.0 0.0 0.0 37.0 67000.0]]
```

RapidMiner **split** operator

Splitting the dataset into the Training set (80%) and Test set (20%)



Python **StandardScaler** operator

Feature Scaling on Training Set & Test Set

```
[ ]  from sklearn.preprocessing import StandardScaler
     sc = StandardScaler()
     X_train[:, 3:] = sc.fit_transform(X_train[:, 3:])
     X_test[:, 3:] = sc.transform(X_test[:, 3:])
```

```
[ ]  print(X_train)
```

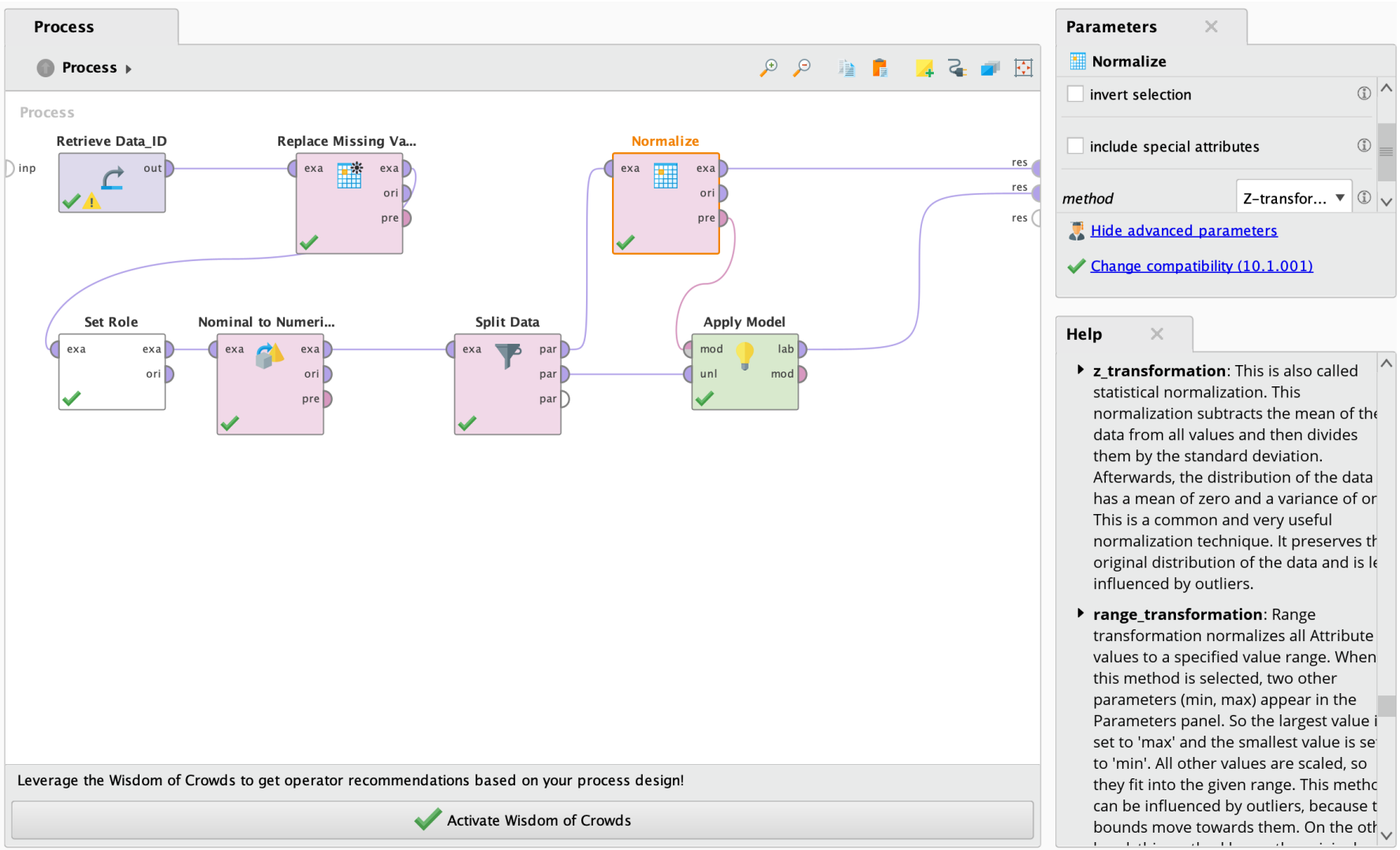
```
[[0.0 0.0 1.0 -0.19159184384578545 -1.0781259408412425]
 [0.0 1.0 0.0 -0.014117293757057777 -0.07013167641635372]
 [1.0 0.0 0.0 0.566708506533324 0.633562432710455]
 [0.0 0.0 1.0 -0.30453019390224867 -0.30786617274297867]
 [0.0 0.0 1.0 -1.9018011447007988 -1.420463615551582]
 [1.0 0.0 0.0 1.1475343068237058 1.232653363453549]
 [0.0 1.0 0.0 1.4379472069688968 1.5749910381638885]
 [1.0 0.0 0.0 -0.7401495441200351 -0.5646194287757332]]
```

```
[ ]  print(X_test)
```

```
[[0.0 1.0 0.0 -1.4661817944830124 -0.9069571034860727]
 [1.0 0.0 0.0 -0.44973664397484414 0.2056403393225306]]
```

RapidMiner **Normalize** operator

Feature Scaling on Training set and Test set



RapidMiner **Normalize** operator with k-NN

Normalize on Training set and Test set

